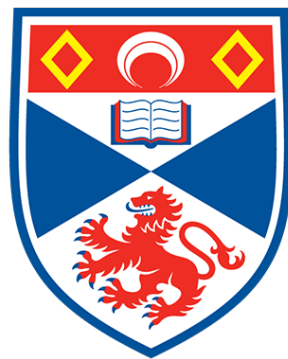


**Preventing Model Collapse in Constrained DDPM Image
Generation of Butterflies
MT4796 Joint Honours Project**

Stella Mortarotti

Supervised by:
Peter Macgregor & Nikolaos Sfakianakis



**University of
St Andrews**

School of Mathematics and Statistics
The University of St Andrews
St Andrews, UK

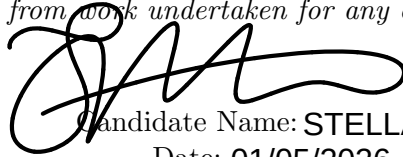
May 1, 2026

Acknowledgements

Many thanks to both of my supervisors, Peter and Nikos, for their support and advice on this project. Thank you to The University of St Andrews School of Computer Science for setting me up with a GPU, and for remaining open 24/7. Thank you to Millie, for reassuring me on initial drafts. Finally, thank you to my incredible and meticulous proofreader. Without Flynn, I would be stuck at

“A butterfly is a tut belfry, trefa butylsi... Fub a tetryl
si, is flutter a by, is a butterfly?”

I certify that this dissertation report has been written by me, is a record of work carried out by me, and is essentially different from work undertaken for any other purpose or assessment.



Candidate Name: STELLA MORTAROTTI

Date: 01/05/2026

Abstract

This study investigates the impact of synthetic data as part of the training corpus on the quality of outputs of a model, designed to generate butterfly images. The model is trained on varying ratios of real to synthetic data, simulating the real-world process of retraining to improve the output quality of the generated images. The study then aims to find an experimental ratio at which the model quality is retained across multiple iterations of training, and the point of model collapse is not reached. This is achieved by using the measures of SSIM, FID, MMD and PCA. It also explores the irregularities observed in the results, and potential improvements to the project to further test the robustness of the hypothesis. This study seeks to contribute to the current debate surrounding the viability of using synthetic data to resolve the scarcity of new data for model training.

Contents

1	Introduction	4
1.1	Existing Research	5
1.2	Contributions	5
2	Mathematical Theory	5
2.1	Problem Set-Up	5
2.2	Forward Diffusion	6
2.2.1	Notes on the Chosen Schedule	8
2.3	Reverse Diffusion	9
2.3.1	Defining the Aims of Reverse Diffusion	9
2.3.2	Deriving the Evidence Function (Stationary Case)	10
2.3.3	Deriving the Evidence Function (Dynamic Case)	11
2.4	Choice of Indicators	16
3	Model Implementation	17
3.1	Dataset	18
3.2	Data Loading	19
3.3	Defining the Scheduler	19
3.4	Training the Model	19
3.5	Sampling the Model	21
4	Provoking Model Collapse	22
4.1	Initial Findings	22
4.2	Calculating Metrics on Initial Results	22
4.3	Further Trials	25
5	Evaluation	26
6	Conclusion	27
A	Dataset Examples	28
B	Timestep and Image size tests	29
C	Scores for SSIM, FID, MMD	29
C.1	FID Results	29
C.2	SSIM Results and Plots	30
C.3	MMD Results and Plots	31
D	PCA Results for 20% intervals of synthetic data	32
E	PCA Results for 5% intervals of synthetic data	34

1 Introduction

In September 2022, OpenAI announced that over 1.5 million users were creating more than 2 million AI-generated images daily [1]. By August 2023, over 15 billion images had been created using Generative AI (Gen AI), with approximately 80% of these created using models based on stable diffusion [2]. Stable diffusion is a technique used by platforms such as stability.ai, that enables high-resolution image generation using latent diffusion models [3], refining the image over time-steps. The rate of adoption has slowed down, but the improvement in the usability, accuracy of the tools, and increased scope of their use will inevitably lead to a greater proportion of the content available on the Internet being created by Gen AI (such data is called **synthetic data**).

Large language models (LLMs) are trained using vast amounts of data. **Crawling** is used to collect this data at such scale, which involves using software to systematically browse and index the World Wide Web (WWW). ChatGPT 3 was trained with 45TB of compressed plaintext (pre-filtering) from the free archive tool CommonCrawl (570GB after filtering) [4]. GPT-5.2 required yet more data; their documentation suggests that OpenAI had to be trained on material other than that available on the open internet, such as synthetic data [5].

Synthetic data is therefore becoming a crucial component of the Gen AI training process, potentially expanding the available training data to compensate for the increasing scarcity of new real data. Studies also suggest that it may reduce the required size of real-world training data by actually boosting the efficiency in Gen AI model training [6].

However, there are drawbacks to adding synthetic data to the training corpus, such as **model collapse**. Herel and Mikolov describe model collapse as “where the generated outputs become severely biased and repetitive” [7]. This implies a serious degradation in the quality of outputs repeatedly generated; “[model collapse] is not a phenomenon within a single training run but is caused by the drift of the training data distribution over generations” [8]. The low-frequency (or high entropy) events of the original dataset are gradually lost over each iteration of training. The resulting output data distribution, apart from being largely unrelated to the original distribution, has very small variance. This is empirically observed as repetitive output images, or where the underlying structure or signal has been lost. Given these properties, the point of model collapse will be evaluated using the measures of **Structural Similarity Index Measure** (SSIM), **Fréchet Inception Distance** (FID), **Maximum Mean Discrepancy** (MMD), and **Principal Component Analysis** (PCA).

This study seeks to evaluate the impact of synthetic data as part of the training corpus on the quality of outputs of a generative model. Firstly, it seeks to demonstrate model collapse in a controlled environment. It then aims to find an experimental ratio at which the model quality is retained across multiple iterations of training.

The model of Gen AI used is a Denoising Diffusion Probabilistic Model (DDPM). These models are studied under the discrete-time, or latent variable model, perspective. They were first introduced by Sohl-Dickstein et al. in 2015 [9] and regained their popularity on refinement by Ho et al. [10]. These models perform better than other Generative Adversarial Networks (GANs) in quality and stability, while remaining relatively tractable [11]. Their underlying dependence on probability distributions guides the choice of indicators used to identify the point of model collapse. A DDPM is constructed following four steps: (1) Selection, (2) Forward Diffusion, (3) Target Generation, (4) Reverse Diffusion. The implementation of the model is done using a Convolutional Neural Network (CNN). Importantly, the minimum viable pipeline (MVP) is relatively fast and of sufficient quality to obtain qualitatively-identifiable outputs. The following sections will provide mathematical formalisations of these steps, as well as their computational implementations.

1.1 Existing Research

The phenomenon of model collapse has been observed empirically across a range of studies in the past five years [12] and described theoretically [13], [14]. Most of these works focus on LLMs such as ChatGPT and Llama, which consume massive datasets, and as a result increasingly rely on poorer quality data such as synthetic data. These studies simulate synthetic reinjection by using “state-of-the-art” generative models, and must use prompts to generate the training data and model outputs [15]. This reduces the control over the exact phenomenon of collapse, as the quality of prompt engineering has an impact on the quality of the output. Other studies focussed on smaller models, but added synthetic images to a large cumulative dataset, on which the model is incrementally trained [16], [17]. This does not allow for direct control on the quality of synthetic data input; instead, the model is permitted to train on data it has already seen, worsening the possible output regardless of the synthetic data. While Dohmatob et al. [13] demonstrated that larger models tend to experience faster rates of model collapse, the training process is far more difficult to replicate empirically on such large models.

1.2 Contributions

The project itself consists of creating a DDPM model, and training it on varying ratios of real to synthetic data. The synthetic data is output from the model at iteration $n - 1$, and used in iteration n of training. The stage of model collapse is then measured using SSIM, FID and MMD, relative to the original data and the first iteration of synthetic data. An estimate for the maximal stable ratio of synthetic data is obtained by using qualitative and quantitative thresholds, and extensions are proposed to further test the universality of this across more iterations and other models.

In summary, the two key focus points of this study are: *How can model collapse be measured?* and *At what proportion of synthetic data reinjection does this happen?* This study seeks to overcome some of the limitations of previous research by reducing the expected output of the generated images. There is no prompt — the model *de facto* outputs images replicating the initial dataset. By using this toy model in a restricted environment, it is possible to circumvent the “training from scratch” paradigm, and incrementally train the data on additional, controlled quantities of synthetic data.

The principal insights from this project are as follows:

- DDPM models improve with the addition of more original data,
- Training on composite datasets with up to 25% synthetic data does not cause model collapse over at least 5 iterations relative to the proposed FID thresholds,
- Model collapse is directly related to the decrease in variance of the generated datasets,
- Under this iterative model, on synthetic reinjection, the decay in output quality is exponential.

2 Mathematical Theory

2.1 Problem Set-Up

This section will present a mathematical formulation of the mechanisms underlying DDPM image generation. From this, it will discuss appropriate indicators that will be used to measure the distribution deviation until point of collapse.

A Denoising Diffusion Probabilistic Model (DDPM) is a generative model that learns to produce new images resembling a training set by approximating its underlying data distribution via a Markov chain [9]. Training consists of two processes: a fixed *forward* process that gradually corrupts a real image by adding Gaussian noise over T time-steps until only pure noise remains, and a learnt *reverse* process in which a neural network is trained to de-noise over the time-steps from T to 0. Then, at inference, the model applies the reverse process to a sample of pure noise, progressively removing noise to generate a new image representative of the original data distribution.

Definition 1 (Pure Noise). *Pure noise is a sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, drawn from a multivariate isotropic Gaussian distribution, containing no recoverable signal from the original data distribution [18].*

Definition 2 (Multivariate Isotropic Gaussian Distribution). *A multivariate isotropic Gaussian is a distribution over $\mathbb{R}^d$ with mean $\mu \in \mathbb{R}^d$ and diagonal covariance matrix $\Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_d^2)$, and is the same as a collection of d independent Gaussian random variables with mean μ_i and variance σ_i^2 , respectively [19].*

2.2 Forward Diffusion

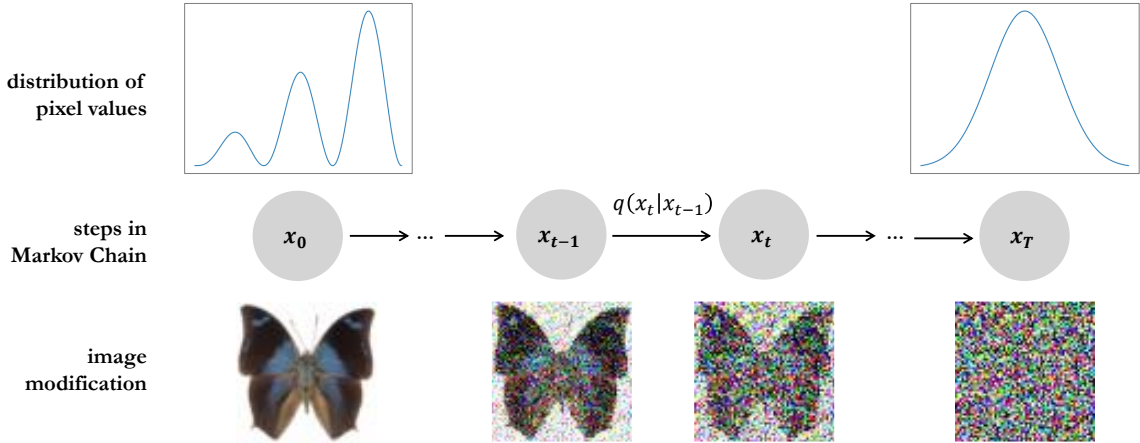


Figure 1: Gradual Addition of Noise to Image in Forward Process

The purpose of **forward diffusion** is to obtain incremental training data for the model. We progress from an original image to one of pure noise, as shown in Figure 1. This is done gradually, across time-steps $t = 0$ to $t = T$.

Consider an initial dataset of real images, each with dimension $(w \times h \times c)$, where w, h, c are the width, height and number of channels respectively¹. In practice, these images are flattened to a vector of length $d = w \cdot h \cdot c$. Then let $q(x_0)$ be the real data distribution over $\mathbb{R}^d$, which is assumed to be multimodal. For each time-step t , a slightly noisier image is obtained, represented by a latent variable $x_t \sim q(x_t|x_{t-1})$. The final distribution of $q(x_T)$ is obtained after T steps of noise addition. It is expected to have unit variance and be unimodal, modelled by a multivariate isotropic Gaussian distribution.

¹An ‘RGB’ image, for example, has $c = 3$ channels, each taking pixel values in $[0, 255]$.

The probability of obtaining the sequence of images $x_0, x_1, \dots, x_T$ can be represented as a Markov chain, where each step is conditionally independent. Using the Markov property, the probability of getting from the starting image x_0 to the image of pure noise x_T is

$$q(x_1 : x_T) = q(x_1, x_2, \dots, x_T | x_0) = q(x_1 | x_0) q(x_2 | x_1) \cdots q(x_T | x_{T-1}) = \prod_{t=1}^T q(x_t | x_{t-1}). \quad (1)$$

The *probabilistic* approach towards forward diffusion suggests that at each step, the original distribution is modified by a chosen **variance schedule** β_t for $t < T$, where $0 < \beta_1 < \beta_2 < \dots < \beta_T < 1$. These β_t are dependent on each step t . Then a single sample is taken from this new Gaussian distribution $\mathcal{N}(x_t; \mu_t, \Sigma_t)$, where $\mu_t = \sqrt{1 - \beta_t} x_{t-1}$, $\Sigma_t = \beta_t I$. The scale factors $\sqrt{1 - \beta_t}$ and $\sqrt{\beta_t}$ are chosen by design to gradually introduce noise [20].

The approach is *deterministic*: we can add noise at each step of a T -step process from image x_0 to image x_T . Suppose ϵ is sampled from another multivariate isotropic Gaussian distribution such that

$$\epsilon_t \sim \mathcal{N}(0, I), t \in (0, T]. \quad (2)$$

Using a scaled version of this noise to add to the (scaled) previous image at each step in the process gives incrementally noisier sample images.

Both these approaches are used to express the Markov chain in tractable form, rather than sampling all images for a large T .

For the simplicity of notation, the hyperparameters are defined as

$$\alpha_t = 1 - \beta_t$$

$$\overline{\alpha}_t = \prod_{i=1}^t \alpha_i.$$

This means that for all time-steps t , it is possible to precompute $\beta_t, \alpha_t, \overline{\alpha}_t$ such that x_t can be sampled at any point for $t < T$.

Theorem 3 (Reparametrisation). *If $X \sim \mathcal{N}(x; \mu, \Sigma)$, then $X = \mu + AZ$, $Z \sim \mathcal{N}(0, I)$, where A is defined using the Cholesky decomposition such that $\Sigma = AA^T$.*

Then it is possible to express x_t by 3 into a deterministic and normal stochastic part, x_t and ϵ_t respectively.

Consider the step $x_0 \rightarrow x_1$ as

$$x_1 = \sqrt{1 - \beta_1} \cdot x_0 + \sqrt{\beta_1} \cdot \epsilon_1 = \sqrt{\alpha_1} \cdot x_0 + \sqrt{1 - \alpha_2} \cdot \epsilon_1. \quad (3)$$

Then the step of $x_1 \rightarrow x_2$ in terms of α_i is

$$x_2 = \sqrt{\alpha_2} \cdot x_1 + \sqrt{1 - \alpha_2} \cdot \epsilon_2. \quad (4)$$

Substituting in the value of x_1 ,

$$\begin{aligned} &= \sqrt{\alpha_2} \cdot (\sqrt{\alpha_1} \cdot x_0 + \sqrt{1 - \alpha_1} \cdot \epsilon_1) + \sqrt{1 - \alpha_2} \cdot \epsilon_2 \\ &= \sqrt{\alpha_1 \alpha_2} \cdot x_0 + \sqrt{(1 - \alpha_1) \alpha_2} \cdot \epsilon_1 + \sqrt{1 - \alpha_2} \cdot \epsilon_2. \end{aligned} \quad (5)$$

Repeating this for some $x_{t-1} \rightarrow x_t$ given $\{x_0, x_1, \dots, x_{t-2}\}$,

$$\begin{aligned}
x_t &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}(\sqrt{\alpha_{t-2}}x_{t-3} + \sqrt{1 - \alpha_{t-2}}\epsilon_{t-3}) + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1 - \alpha_t}\epsilon_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}\alpha_{t-2}}x_{t-3} + \sqrt{\alpha_t\alpha_{t-1}(1 - \alpha_{t-2})}\epsilon_{t-3} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1}. \quad (6)
\end{aligned}$$

Theorem 4 (Addition of Gaussian variables). *Given two Gaussian variables $X \sim \mathcal{N}(\mu_1, \sigma_1^2)$, $Y \sim \mathcal{N}(\mu_2, \sigma_2^2)$, the sum of independent normal distributions is also a Gaussian distribution, such that*

$$X + Y \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2).$$

Consider then the ϵ terms in (6). These are all scaled Gaussian variables, from an original distribution where $\mu = 0, \Sigma = I$.

$$\begin{aligned}
\sqrt{\alpha_t\alpha_{t-1}(1 - \alpha_{t-2})}\epsilon_{t-3} &\implies X \sim \mathcal{N}(0, \alpha_t\alpha_{t-1}(1 - \alpha_{t-2}) \cdot I) \\
\sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} &\implies Y \sim \mathcal{N}(0, \alpha_t(1 - \alpha_{t-1}) \cdot I) \\
\sqrt{1 - \alpha_t}\epsilon_{t-1} &\implies Z \sim \mathcal{N}(0, (1 - \alpha_t) \cdot I).
\end{aligned}$$

Therefore by Theorem 4,

$$\begin{aligned}
\sigma_X^2 + \sigma_Y^2 + \sigma_Z^2 &= \alpha_t\alpha_{t-1}(1 - \alpha_{t-2}) + \alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) \\
&= 1 - \alpha_t\alpha_{t-1}\alpha_{t-2} \\
&\implies X + Y + Z \sim \mathcal{N}(0, (1 - \alpha_t\alpha_{t-1}\alpha_{t-2}) \cdot I).
\end{aligned}$$

Repeating this process recursively for ϵ , the sum of all stochastic terms from 0 to T becomes

$$\epsilon_t^* \sim \mathcal{N}(0, (1 - \bar{\alpha}_t) \cdot I)$$

and the process $x_0 \rightarrow x_t$ can be written as the final closed form equation of

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \epsilon_t^*. \quad (7)$$

2.2.1 Notes on the Chosen Schedule

The chosen variance schedule should allow for the total accumulated variance to reach 1, and the mean of the distribution to tend towards 0; at each step $t < T$, the distribution is scaled with the final goal of reaching an isotropic Gaussian distribution. Research by Nichol et al. [21] demonstrates that the use of a cosine schedule is most effective for DDPM. This schedule allows for gradual addition of noise that slows towards the extremes of $t = 0$ and $t = T$. The comparative practical advantage of this schedule over a more traditional linear schedule can be seen in Figure 3.

Using the cosine schedule, the actual expressions for β_t from $\bar{\alpha}_t$ can be derived as

$$\begin{aligned}
f(t) &= \cos\left(\frac{\frac{t}{T} + s}{1 + s} \cdot \frac{\pi}{2}\right)^2 \implies f(0) = \cos\left(\frac{s}{1 + s} \cdot \frac{\pi}{2}\right)^2, \\
\bar{\alpha}_t &= \frac{f(t)}{f(0)},
\end{aligned} \quad (8)$$

such that

$$\beta_t = 1 - \frac{\overline{\alpha}_t}{\overline{\alpha}_{t-1}} = 1 - \frac{f(t)}{f(t-1)}.$$

The fixed offset s is a calibration parameter, deliberately shifting the cosine curve to flatten it near $t = 0$. Recall that the variance of the distributions considered in the forward process is $\sigma^2 = \beta_t$. The standard deviation $\sigma = \sqrt{\beta_t}$ near $t = 0$ must be sufficiently small such that the model initially learns from relatively non-noisy images. As such, in the context of diffusion models, $\sqrt{\beta_0}$ must be smaller the meaningful distance between pixels (given as the normalised range of values divided by the pixel values), ensuring a controlled addition of noise and preventing too much information loss [21], [22]. The experimental value of s in (8) is given in the later section 3.3.

2.3 Reverse Diffusion

The aim of reverse diffusion is to start from an image of pure noise (the targets from the forward process), and learn trajectories that could return to the original distribution of real images. All the images generated at intermediary steps during forward diffusion are used to learn the path from $x_T \rightarrow x_0$.² The ‘perfect’ path is given by $q(x_{t-1}|x_t)$, and is the reverse of the path in forward diffusion!

It is not possible to explicitly identify every path from pure noise to a more realistic image due to the exponential, intractable nature of the possible routes. Indeed, the ‘perfect’ denoising process $q(x_{t-1}|x_t)$ remains intractable, but can be approximated using $p_\theta(x_{t-1}|x_t)$ as some *learnt* Gaussian distribution. By using the mathematical derivations of the forward process, it is possible to explicitly define a tractable loss function which can be optimised using a neural network. As such, the neural network will approximate a conditional probability distribution for the reverse process, $p_\theta(x_{t-1}|x_t)$, where θ are parameters of the neural network, updated by gradient descent³ [24].

In summary, the goal of reverse diffusion is to learn to generate realistic images, resembling those in the original distribution. To do this, ideally the original distribution $q(x_{t-1}|x_t)$ would be known. As this is impossible to find, it is approximated by $p_\theta(x_{t-1}|x_t)$. The goal is to find the parameters θ that make the two distributions as close as possible.

2.3.1 Defining the Aims of Reverse Diffusion

Suppose an original image x_0 is taken from the initial distribution $q(x_0)$, with the progressively noised latent variables $x_{1:T} \equiv (x_1, x_2, \dots, x_T)$ learnt during forward diffusion. These can be modelled using a joint probability distribution $p_\theta(x_{0:T})$. The goal then is to maximise the log-likelihood $\ln p_\theta(x_0)$ over the observed data x_0 (this is equivalent to the model recognising that the original image is indeed a real, representative image of the original distribution). Finding $p_\theta(x_0)$ requires marginalising out the latent variables as

$$p(x_0) = \int p(x_{0:T}) dx_{1:T} \quad (9)$$

— this technique applied to all latent variables is intractable for large T as it would force us to integrate over very high dimensionality spaces (note that for an RGB image of size m ,

²Using vector theory, the trajectories represent the ability to navigate from any point in the n -dimensional field to the manifold of that field.

³**Gradient descent** is a type of optimisation algorithm that minimises a loss function by attempting to learn the gradient of the function locally [23].

the space is already of dimensions $m \times m \times 3$). For this reason, the approximate posterior $p_\theta(x_{1:T}|x_0)$ is also intractable.

Definition 5 (Evidence function).

$$\ln p_\theta(x_0) = \mathcal{L}(\phi, \theta; x_0) + D_{KL}(q(x_{1:T}|x_0)||p_\theta(x_{1:T}|x_0)). \quad (10)$$

The evidence function describes the log-likelihood of an observation (x_0) over the entire real distribution, and is intractable. It is composed of the Evidence Lower Bound (ELBO) term $\mathcal{L}$ and Kullback-Leibler Divergence D_{KL} . This equation proposed by Ho et al. [10] will be derived fully in the next section.

Definition 6 (Evidence Lower Bound (ELBO)).

$$\mathcal{L}(\phi, \theta; x_0) := \mathbb{E}_{x_{1:T} \sim q(x_{1:T}|x_0)} [\ln \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}]. \quad (11)$$

The ELBO term converts the marginal likelihood into expectation [25]. It is typically this term that is maximised in neural networks, as it is a tractable lower bound for the intractable evidence function. In other words, since it is impossible to directly calculate the likelihood of an image, or set of images, the goal is to maximise a lower bound that is computationally tractable.

Definition 7 (Kullback-Leibler (KL) loss term).

$$D_{KL}(q(x_{1:T}|x_0)||p_\theta(x_{1:T}|x_0)) = \mathbb{E}_{x_{1:T} \sim q(x_{1:T}|x_0)} [\ln \frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T}|x_0)}]. \quad (12)$$

The Kullback-Leibler (KL) loss term provides an information-theoretic perspective on how much information is lost if one probability distribution is used to approximate another. This is also called the relative entropy [25], and measures the distance between two distributions [26]. An equivalent goal during learning is to minimise the KL divergence between the model's approximation of $p_\theta(z|x_0)$ and the true value $q(z|x_0)$, or the difference between the predicted noise and the actual noise introduced in the forward step. Note that $D_{KL} > 0$ always, and if $q = p_\theta$, $D_{KL} = 0$.

The following sections derive the final loss function to be optimised in terms of D_{KL} by

1. Proving the derivation of the basic evidence function (10),
2. Applying this function to a temporal sequence (in this case, from $t = 0$ to $t = T$),
3. Decomposing the expanded evidence function into individual time-step losses, so that the reverse process becomes computationally tractable.

2.3.2 Deriving the Evidence Function (Stationary Case)

Theorem 8 (Chain Rule of Probability). *The chain rule of probability states that*

$$p(x_{0:T}) = p(x_0)p(x_{1:T}|x_0). \quad (13)$$

To derive the evidence lower bound as given in Equation 10, start by considering the conditional probability distribution $q(x_{1:T}|x_0)$ over $(x_1, \dots, x_T)$, such that by the axiom of total probability,

$$1 = \int q(x_{1:T}|x_0) dx_{1:T}. \quad (14)$$

Given that the aim is to approximate the model posterior distribution $p_\theta(x_0)$ from its prior distribution, multiply both sides of (14) by $\ln p_\theta(x_0)$ such that

$$\begin{aligned} \ln p_\theta(x_0) &= \ln p_\theta(x_0) \int q(x_{1:T}|x_0) dx_{1:T} \\ &= \int q(x_{1:T}|x_0) \ln p_\theta(x_0) dx_{1:T} \\ &= \int q(x_{1:T}|x_0) \ln \left(\frac{p_\theta(x_{0:T})}{p_\theta(x_{1:T}|x_0)} \right) dx_{1:T} && \text{by Theorem 8} \\ &= \int q(x_{1:T}|x_0) \ln \left(\frac{p_\theta(x_{0:T}) \cdot q(x_{1:T}|x_0)}{p_\theta(x_{1:T}|x_0) \cdot q(x_{1:T}|x_0)} \right) dx_{1:T} \\ &= \int q(x_{1:T}|x_0) \ln \left(\frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right) dx_{1:T} + \int q(x_{1:T}|x_0) \ln \left(\frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T}|x_0)} \right) dx_{1:T} \\ &= \mathbb{E}_{x_{1:T} \sim q(x_{1:T}|x_0)} [\ln \left(\frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)} \right)] + \mathbb{E}_{x_{1:T} \sim q(x_{1:T}|x_0)} [\ln \left(\frac{q(x_{1:T}|x_0)}{p_\theta(x_{1:T}|x_0)} \right)] && \text{by } \mathbb{E}_{x \sim p(x)}[x] = \int xp(x)dx \\ &= \mathcal{L}(\phi, \theta; x_0) + D_{KL}(q(x_{1:T}|x_0) || p_\theta(x_{1:T}|x_0)), \end{aligned}$$

retrieving the evidence function (10) as in the definition. Although the posterior distribution $p_\theta(x_{1:T}|x_0)$ is not defined; by using the non-negativity of the KL-divergence term, one can generate the inequality

$$\ln p_\theta(x_0) \geq \mathcal{L}(\phi, \theta; x_0). \quad (15)$$

This inequality precisely states the purpose of the ELBO in providing a lower bound to the intractable log-likelihood term.

2.3.3 Deriving the Evidence Function (Dynamic Case)

As discussed in the introduction to Section 2.3, the true posterior distribution q is approximated by a parametrised distribution $p_\theta(x_{t-1}|x_t)$, where p_θ is assumed to be Gaussian, $p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$. The aim of this section is to explicitly derive parametrised expressions for $\mu_\theta, \Sigma_\theta$ given the time-steps of the latent variables. This involves expanding the initial evidence function (10) under the temporal sequence. Then, the ELBO term is decomposed into individual time-step losses (L_t), so that the neural network can de-noise at any stage of the reverse process.

Considering the factorised form of the Markov property for the reverse process,

$$p_\theta(x_{0:T}) = p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t),$$

and using the factored form of the forward process in (1), substituting these factorisations into the ELBO equation:

$$\begin{aligned}
\mathcal{L}(\phi, \theta; x_0) &= \mathbb{E}_{q(x_{1:T}|x_0)} \left[\ln \left(\frac{p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)}{\prod_{t=1}^T q(x_t|x_{t-1})} \right) \right] \\
&= \mathbb{E}_{q(x_{1:T}|x_0)} \left[\ln p_\theta(x_T) + \sum_{t=1}^T (\ln p_\theta(x_{t-1}|x_t) - \ln q(x_t|x_{t-1})) \right] \\
&= \mathbb{E}_{q(x_{1:T}|x_0)} \left[\ln p_\theta(x_T) + \ln p_\theta(x_0|x_1) - \ln q(x_1|x_0) + \sum_{t=2}^T (\ln p_\theta(x_{t-1}|x_t) - \ln q(x_t|x_{t-1})) \right].
\end{aligned} \tag{16}$$

Theorem 9 (Bayes' Theorem). *Given a prior distribution $p_\theta(x_{1:T})$, the likelihood term $p_\theta(x_0|x_{1:T})$ and an observation $p_\theta(x_0)$, the global posterior is given by [27]*

$$p_\theta(x_{1:T}|x_0) = \frac{p_\theta(x_0|x_{1:T})p_\theta(x_{1:T})}{p_\theta(x_0)}. \tag{17}$$

Theorem 9 can be applied to the *local* posterior as

$$\begin{aligned}
q(x_{t-1}|x_t, x_0) &= \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
\implies q(x_t|x_{t-1}) &= q(x_{t-1}|x_t, x_0) \cdot \frac{q(x_t|x_0)}{q(x_{t-1}|x_0)} \\
\implies \ln q(x_t|x_{t-1}) &= \ln q(x_{t-1}|x_t, x_0) + \ln q(x_t|x_0) - \ln q(x_{t-1}|x_0),
\end{aligned} \tag{18}$$

and substituting this into Equation 16, gives

$$\begin{aligned}
\mathcal{L}(\phi, \theta; x_0) &= \mathbb{E}_{q(x_{1:T}|x_0)} [\ln p_\theta(x_T) + \ln p_\theta(x_0|x_1) - \ln q(x_1|x_0)] \\
&\quad + \sum_{t=2}^T (\ln p_\theta(x_{t-1}|x_t) - [\ln q(x_{t-1}|x_t, x_0) + \ln q(x_t|x_0) - \ln q(x_{t-1}|x_0)]) \\
&= \mathbb{E}_{q(x_{1:T}|x_0)} [\ln p_\theta(x_T) + \ln p_\theta(x_0|x_1) - \ln q(x_1|x_0) + \sum_{t=2}^T (\ln \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} - \ln q(x_t|x_0) + \ln q(x_{t-1}|x_0))] \\
&= \mathbb{E}_{q(x_{1:T}|x_0)} [\ln p_\theta(x_T) + \ln p_\theta(x_0|x_1) - \ln q(x_1|x_0) + \underbrace{(\ln q(x_1|x_0) - \ln q(x_T|x_0))}_{\text{by telescoping sum}} + \sum_{t=2}^T \ln \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}] \\
&= \mathbb{E}_{q(x_{1:T}|x_0)} [\ln p_\theta(x_T) - \ln q(x_T|x_0)] + \mathbb{E}_{q(x_{1:T}|x_0)} [\ln p_\theta(x_0|x_1)] + \mathbb{E}_{q(x_{1:T}|x_0)} \left[\sum_{t=2}^T \ln \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right],
\end{aligned}$$

which can be rearranged to reach the final DDPM term found in Ho et al. [10] that is optimised during training, using the same relation as in (15):

$$\ln p_\theta(x_0) \geq \underbrace{\mathbb{E}_{q_\phi(x_1|x_0)} [\ln p_\theta(x_0|x_1)]}_{\text{reconstruction term } L_1} - \underbrace{D_{KL}(q_\phi(x_T|x_0)||p_\theta(x_T))}_{\text{prior matching term } L_0} - \sum_{t=2}^T \underbrace{\mathbb{E}_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))]}_{\text{denoising matching term } L_{t-1}}.$$

The L_0 term can be estimated using **Monte Carlo** methods⁴.

⁴This is a class of algorithms which use random sampling to produce approximate solutions, for instance by evaluating expectations as simple averages [28].

The L_T term is constant in the loss function, and represents how close the ‘noisified’ input is to the prior standard Gaussian.

The L_{t-1} term is the denoising term to be optimised. If $q(x_{t-1}|x_t, x_0)$ can be understood as the ‘perfect’ denoising process, which sees both the gradually noisier images and the perfect image x_0 (end-goal of the reverse diffusion), and $p_\theta(x_{t-1}|x_t)$ is the model’s actual de-noiser, then the goal will be to reduce the KL-divergence between the perfect and actual de-noisers.

Definition 10 (KL loss term for multivariate normal distributions). *Using a reformulation of the KL-term in (12) written in terms of two distributions,*

$$q(x_{t-1}|x_t, x_0) \sim \mathcal{N}(x; \mu, \Sigma), \quad p_\theta(x_{t-1}|x_t) \sim \mathcal{N}(x; \mu_\theta, \Sigma_\theta),$$

where d is the dimension of both covariance matrices [29],

$$D_{KL}(q||p_\theta) = \frac{1}{2} \left[\ln \frac{\Sigma_\theta}{\Sigma} - d + \text{tr}(\Sigma_\theta^{-1} \Sigma) + (\mu_\theta - \mu)^T \Sigma_\theta^{-1} (\mu_\theta - \mu) \right]. \quad (19)$$

To apply (19), it is necessary therefore to derive expressions for both the mean and variance of the distributions. During reverse diffusion, the starting Gaussian distributions from which x_T are sampled are multivariate and isotropic with diagonal covariance matrix $\Sigma_\theta(x, t) = \sigma^2 I$. From this assumption, the distributions can be studied as a product of d univariate Gaussians.

Then, consider the univariate **probability density function** (PDF), rather than the multivariate analogue, given as

$$p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right). \quad (20)$$

Rewriting Equation 18 using the tractable forms derived in the forward process,

$$q(x_{t-1}|x_t, x_0) = \frac{\mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t) \cdot I) \mathcal{N}(x_{t-1}; \sqrt{\alpha_{t-1}}x_0, (1 - \alpha_{t-1}) \cdot I)}{\mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t) \cdot I)}. \quad (21)$$

Approximating (21) with the Gaussian PDF,

$$\begin{aligned}
q(x_{t-1}|x_t, x_0) &\propto \exp \left(- \left[\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{2(1-\alpha_t)} + \frac{(x_{t-1} - \sqrt{\alpha_{t-1}}x_0)^2}{2(1-\alpha_{t-1})} - \frac{(x_t - \sqrt{\alpha_t}x_0)^2}{2(1-\alpha_t)} \right] \right) \\
&= \exp \left(-\frac{1}{2} \left[\frac{x_t^2 - 2\sqrt{\alpha_t}x_tx_{t-1} + \alpha_tx_{t-1}^2}{(1-\alpha_t)} + \frac{x_{t-1}^2 - 2\sqrt{\alpha_{t-1}}x_0x_{t-1} + \alpha_{t-1}x_0^2}{(1-\alpha_{t-1})} \right. \right. \\
&\quad \left. \left. - \frac{x_t^2 - 2\sqrt{\alpha_t}x_tx_0 + \alpha_tx_0^2}{(1-\alpha_t)} \right] \right) \\
&= \exp \left(-\frac{1}{2} \left[\frac{-2\sqrt{\alpha_t}x_tx_{t-1} + \alpha_tx_{t-1}^2}{(1-\alpha_t)} + \frac{x_{t-1}^2 - 2\sqrt{\alpha_{t-1}}x_0x_{t-1}}{(1-\alpha_{t-1})} \right] + C(x_0, x_t) \right) \\
&\propto \exp \left(-\frac{1}{2} \left[\frac{-2\sqrt{\alpha_t}x_tx_{t-1} + \alpha_tx_{t-1}^2}{(1-\alpha_t)} + \frac{x_{t-1}^2 - 2\sqrt{\alpha_{t-1}}x_0x_{t-1}}{(1-\alpha_{t-1})} \right] \right) \\
&= \exp \left(-\frac{1}{2} \left[\left(\frac{\alpha_t}{1-\alpha_t} + \frac{1}{1-\alpha_{t-1}} \right) x_{t-1}^2 - 2 \left(\frac{\sqrt{\alpha_t}x_t}{(1-\alpha_t)} + \frac{\sqrt{\alpha_{t-1}}x_0}{(1-\alpha_{t-1})} \right) x_{t-1} \right] \right) \\
&= \exp \left(-\frac{1}{2} \left[\frac{1-\alpha_t}{(1-\alpha_t)(1-\alpha_{t-1})} x_{t-1}^2 - 2 \frac{\sqrt{\alpha_t}(1-\alpha_{t-1})x_t + \sqrt{\alpha_{t-1}}(1-\alpha_t)x_0}{(1-\alpha_t)(1-\alpha_{t-1})} x_{t-1} \right] \right) \\
&= \exp \left(-\frac{1}{\frac{2(1-\alpha_t)(1-\alpha_{t-1})}{1-\alpha_t}} \left[x_{t-1}^2 - \frac{2}{1-\alpha_t} (\sqrt{\alpha_t}(1-\alpha_{t-1})x_t + \sqrt{\alpha_{t-1}}(1-\alpha_t)x_0) x_{t-1} \right] \right).
\end{aligned}$$

The expression is now of the form $\exp\left(-\frac{1}{2\sigma^2}(x^2 - 2\mu x)\right)$, identifying μ and σ^2 :

$$\Rightarrow q(x_{t-1}|x_t, x_0) \propto \mathcal{N} \left(x_{t-1}; \frac{\sqrt{\alpha_t}(1-\alpha_{t-1})x_t + \sqrt{\alpha_{t-1}}(1-\alpha_t)x_0}{1-\alpha_t}, \frac{(1-\alpha_t)(1-\alpha_{t-1})}{1-\alpha_t} \cdot I \right).$$

The variance of the distribution p_θ for approximation is

$$\Sigma = \frac{(1-\alpha_t)(1-\alpha_{t-1})}{1-\alpha_t}. \tag{22}$$

The mean is currently in terms of both an original image x_0 , and intermediate steps. This

expression must be only in terms of x_t : substituting in the equation for x_0 as rearranged from (7)

$$\begin{aligned}
\mu(x_t, x_0) &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})x_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0}{1 - \bar{\alpha}_t} \\
&= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})x_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t}{\sqrt{\bar{\alpha}_t}}}{1 - \bar{\alpha}_t} \\
&= \frac{\alpha_t(1 - \bar{\alpha}_{t-1})x_t + (1 - \alpha_t)(x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t)}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} \\
&= \frac{-\alpha_t\bar{\alpha}_{t-1}x_t + x_t - \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t + \alpha_t\sqrt{1 - \bar{\alpha}_t} \cdot \epsilon_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} \\
&= \frac{1 - \bar{\alpha}_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)}x_t - \frac{\sqrt{1 - \bar{\alpha}_t} - \alpha_t\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} \cdot \epsilon_t \\
&= \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \cdot \epsilon_t) \\
\Rightarrow \mu(x_t, t) &= \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \cdot \epsilon_t) = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \cdot \epsilon_t).
\end{aligned}$$

This is the closed-form prediction of μ and Σ as discussed in Nichol et al. [21] and Ho et al. [10] as part of their sampling algorithm.

In the loss function, the final aim is to minimise this D_{KL} value. In training, the variance of both (real and approximate) distributions are kept constant, and the only aim is to learn the mean for simplicity [21]. The relation for the two multivariate normal distributions can thus be written as:

$$\begin{aligned}
D_{KL} = (19) &= \operatorname{argmin}_{\theta} \frac{1}{2} [\ln \frac{\Sigma}{\Sigma} - d + \operatorname{tr}(\Sigma^{-1}\Sigma) + (\mu_{\theta} - \mu)^T \Sigma^{-1}(\mu_{\theta} - \mu)] \\
&= \operatorname{argmin}_{\theta} \frac{1}{2} [\ln 1 - d + d + (\mu_{\theta} - \mu)^T \Sigma^{-1}(\mu_{\theta} - \mu)] \\
&= \operatorname{argmin}_{\theta} \frac{1}{2} [(\mu_{\theta} - \mu)^T (\sigma^2 \cdot I)^{-1}(\mu_{\theta} - \mu)] \\
&= \operatorname{argmin}_{\theta} \frac{1}{2} [(\mu_{\theta} - \mu)^T (\sigma^2 \cdot I)^{-1}(\mu_{\theta} - \mu)] \\
&= \operatorname{argmin}_{\theta} \frac{1}{2\sigma^2} \|\mu_{\theta} - \mu\|_2^2 \\
&= \operatorname{argmin}_{\theta} \frac{1}{2\sigma^2} \left\| \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \cdot \epsilon_{\theta}) - \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \cdot \epsilon) \right\|_2^2 \\
&= \operatorname{argmin}_{\theta} \frac{1}{2\sigma^2} \left\| \frac{\beta_t}{\sqrt{\alpha_t}\sqrt{1 - \bar{\alpha}_t}}(\epsilon - \epsilon_{\theta}) \right\|_2^2 \\
&= \operatorname{argmin}_{\theta} \frac{1}{2\sigma^2} \frac{(1 - \alpha_t)^2}{\alpha_t(1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_{\theta}\|_2^2 \\
&= \operatorname{argmin}_{\theta} \frac{1}{2\sigma^2} \frac{\beta_t^2}{\alpha_t(1 - \bar{\alpha}_t)} \left\| (\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon), t) \right\|_2^2.
\end{aligned}$$

The formula above is the exact expression that is optimised in the implementation. In practice, the reverse diffusion step is actually carried out using a **Convolutional Neural Net** (CNN-UNet) which removes noise gradually. Essentially, it predicts noise added from $x_{t-1} \rightarrow x_t$

by using the minimised mean squared error as a scaled noise prediction: this is the difference between the actual noise term ϵ and the predicted noise term ϵ_θ .

2.4 Choice of Indicators

There are two aspects of model collapse that are measured in this project: the first is the distance between the former data distribution and that of the generated data. The second is the collapse in the variance of the distribution of the generated outputs. Such behaviours can be quantified using the following metrics⁵ SSIM, FID, MMD and PCA.

Definition 11 (SSIM). *For a patch centred at (x, y) the SSIM for two images A, B is*

$$\text{SSIM}((x, y)_A, (x, y)_B) = \underbrace{\left[\frac{2\mu_A\mu_B + C_1}{\mu_A^2 + \mu_B^2 + C_1} \right]^\alpha}_{\text{luminance}} \underbrace{\left[\frac{2\sigma_A\sigma_B + C_2}{\sigma_A^2 + \sigma_B^2 + C_2} \right]^\beta}_{\text{contrast}} \underbrace{\left[\frac{\sigma_{AB} + C_3}{\sigma_A\sigma_B + C_3} \right]^\gamma}_{\text{structure}} \quad (23)$$

where $\mu_A = \frac{1}{n} \sum_{i=1}^n A_i$, given A_i are the n pixel values in a given patch of A , and equivalently for B [30].

This is the most common indicator used in industry to understand the underlying structure of images. SSIM is mathematically tractable unlike the two other indicators used here, and only considers the *luminance*, *contrast* and *structure* per image. In practice, $\alpha = \beta = \gamma = 1$. SSIM does not compute one global mean for the whole image, but usually works across small patches, typically 11×11 windows.

Definition 12 (Mean SSIM). *The mean SSIM ($m\text{SSIM}$) taken across the entire image is given for an image of dimensions (w, h) as*

$$m\text{SSIM}(A, B) = \frac{1}{wh} \sum_{x,y} \text{SSIM}(x, y). \quad (24)$$

Nilsson et al. [30] show that SSIM can produce non-intuitive results, where noise occasionally increases the SSIM value due to the bias towards mean and variance (brightness and contrast). $m\text{SSIM}$ can to some extent replicate human perception: images need not match-pixel wise, but may still ‘look right’ structurally.

Definition 13 (FID). *Suppose there are two variables: a synthetic $Y = \{y_i\}_{i=1}^m$ and a real $X = \{x_i\}_{i=1}^n$. An Inception network defines a feature map $f : \chi \rightarrow \mathbb{R}^d, \chi \in \{X, Y\}$, forming two sets of feature vectors $\mathbf{w}_i = f(x_i), \mathbf{z}_i = f(y_i)$ within the finite-dimension Euclidean space $\mathbb{R}^d$. Fitting Gaussians to the obtained feature clouds, the mean μ and variance Σ for features $\mathbf{w}_i$ (and equivalently for $\mathbf{z}_i$) can be understood as*

$$\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{w}_i, \quad \Sigma = \frac{1}{n-1} \sum_{i=1}^n (\mathbf{w}_i - \mu)(\mathbf{w}_i - \mu)^\top.$$

Then the FID, given the two fitted multidimensional Gaussian distributions $\mathbf{w}_i \sim \mathcal{N}(\mu, \Sigma)$ and $\mathbf{z}_i \sim \mathcal{N}(\mu', \Sigma')$, is the 2-Wasserstein distance [31]

$$d_F(\mathcal{N}(\mu, \Sigma), \mathcal{N}(\mu', \Sigma'))^2 = \|\mu - \mu'\|_2^2 + \text{tr}(\Sigma + \Sigma' - 2(\Sigma^{\frac{1}{2}}\Sigma'\Sigma^{\frac{1}{2}})^{\frac{1}{2}}).$$

⁵Note that the term ‘metric’ is not used in the strict mathematical sense here – these indicators do not hold the triangle inequality, for instance.

FID is used to measure the distance between the generated data and the original distribution. It represents how far the synthetic data distribution diverges from the real data by using Inception features to represent higher-level visual structure, rather than pixel-level information [32]. This is done by using the late layers (embeddings) of learning of a small CNN. The usual implementation takes 2,048 activations by default, but can also take 64, 192 and 768 features for smaller resolution or less complex images. The higher the FID score, the greater the drift in synthetic distribution, relative to the original data or a first synthetic iteration. Equivalently, the lower the FID score, the closer the synthetic variable Y is to some real variable X .

Definition 14 (MMD).

$$\text{MMD}(P, Q) \approx \frac{1}{n(n-1)} \sum_{i=1}^n \sum_{j=1, j \neq i}^n k(\mathbf{w}_i, \mathbf{w}_j) - \frac{2}{n^2} \sum_{i=1}^n \sum_{j=1}^n k(\mathbf{w}_i, \mathbf{z}_j) + \frac{1}{n(n-1)} \sum_{i=1}^n \sum_{j=1, j \neq i}^n k(\mathbf{z}_i, \mathbf{z}_j),$$

where n is the batch size, $\mathbf{w}_i \sim P = \mathcal{N}(\mu, \Sigma)$ and $\mathbf{z}_i \sim Q = \mathcal{N}(\mu', \Sigma')$, and $k(\mathbf{x}, \mathbf{y}) = \exp(-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2})$ is the Gaussian radial basis function kernel.

MMD (later implemented as **Kernel Inception Distancem** (KID) [33]), compares two distributions by sampling feature vectors from two distributions. KID essentially computes the MMD for an entire batch and takes the average.

Definition 15 (PCA). *PCA is a dimensionality reduction technique which transforms the data into a new coordinate system such that the axes – called **principal components** – are ordered by the amount of variance they explain [27].*

In the context of image processing, a d -dimensional distribution is reduced into 2 dimensions, representing those of highest variance. PCA is a particularly relevant metric to consider in this project, where the collapse in variance of the underlying distribution is a potential indicator for collapse. In this project, images are of size 64×64 with 3 channels for RGB, so there are 12,288 features per image. PCA then is carried out as follows:

1. Flattening these images, for n images we get a feature matrix M of shape $(n \times 12288)$.
2. Subtract the mean image (average pixel-by-pixel) from the feature matrix to give normalised matrix X .
3. Compute the covariance matrix $C = \frac{1}{N} \cdot X^\top X$ of shape (12288×12288) .
4. Retrieve the 2 largest eigenvalues of C as v_1, v_2 .
5. Project all images onto PC1 and PC2 by $Z = X \cdot v_i^\top$, returning $(n, 2)$, or the set of coordinates in 2D-PCA space.

These points can then be plotted to form a feature cloud and allowing visual observation of the variance. Points in a cluster have similar features, thus the more dense the cluster, the lower the variance of the underlying data (particularly because the *highest* variance is captured by these PCA plots).

All four of these metrics will be considered in combination to assess the degree of divergence of the synthetic distribution, and evaluate a point of model collapse.

3 Model Implementation

The basic structure of the iterative model is shown in Figure 2. The three key features of this model are as follows. Firstly, training is cumulative over each iteration, not ‘from scratch’.

Secondly, a certain **ratio** of synthetic data, output from the previous round, is injected at the next round of training. Thirdly, the underlying dataset is not cumulative, but new and distinct on each round of training: the real data is from a new split, and the synthetic data is exclusively from the immediate previous round.

The structure of the model and the processes of data injection for varying proportions of synthetic data is described below in more detail.

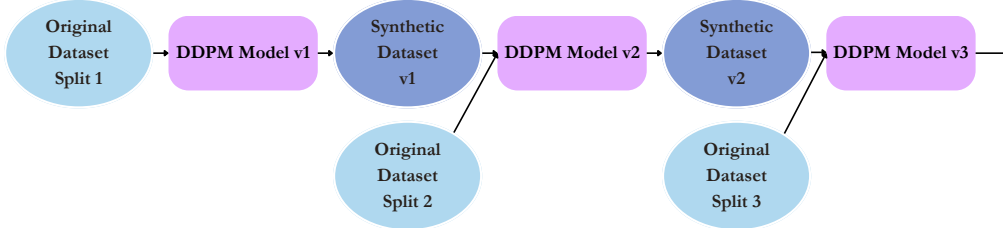


Figure 2: Model for synthetic data reinjection (adapted from [16])

3.1 Dataset

The choice of an appropriate dataset is largely dependent on consistency and size. Firstly the model, being an MVP of stable diffusion, is only trained on 1000 images per training iteration, which is not large by Gen-AI corpus standards. If there is too much variance in the initial dataset, the quality of output is poor even on the first iteration. Secondly, the dataset has to be large enough to have ‘hold-out’ splits. These unseen images are used during future iterations of the model training in addition to synthetic images. The additional images should also fit the variance of the initial data set to prevent the observations being a result of **model drift** (“the gradual decline in a model’s performance after deployment due to changes in the environment” [8]).

The dataset used is the full Smithsonian butterfly dataset from [34], with significant cleaning, reducing the set from 9,213 images to 6,578. To obtain outputs that qualitatively resemble butterflies, it is necessary to remove poor quality images that do not represent a typical butterfly accurately, or contain rulers and other distracting material. The four steps of dataset cleaning are:

1. Background removal using the powerful `dis_bg_remover` package from [35],
2. Pasting the retrieved image onto a white background using `cv2`,
3. Text detection using `pytesseract` and deletion of images containing non-whitelisted characters,
4. Manual verification and reclassification of some results

The resulting dataset can be found at [36]. Some sample images are given in Appendix A.

Note that it would be possible to carry out the experiment on a dataset that has not been filtered and formatted so rigorously. From a statistical perspective, the original dataset likely has a higher variance, perhaps slowing the model decay towards collapse or making a more marked transition into collapsed state. However, for the sake of qualitative analysis of the model’s outputs, it is beneficial to work with a clean and consistent corpus.

3.2 Data Loading

A base dataset is defined at the start of each experiment. Of this, 1000 real images are used to train a base model. The rest of the loaded dataset is used during the next training iterations as a training dataset. The number of splits is determined by the number of images needed per split. Each training split of real images contains sufficient images to complement the synthetic reinjection, and to create a total composite split of 1000 images.

Data augmentation is “a technique to increase the diversity of your training set by applying random (but realistic) transformations” [37]. This pipeline uses an implementation of the `transforms.Compose` package:

1. Resize: setting the image size to 64 pixel square across the entire dataset,
2. Random Horizontal Flip: randomly flipping data horizontally (i.e. across the vertical axis).
3. To Tensor: takes the PIL image and converts it to a tensor with appropriately scaled values.
4. Normalize: map the original tensor value of $[0, 1]$ to a normalized map of $[-1, 1]$. This is done by taking parameters of $\mu = 0.5$ and $\sigma = 0.5$. The calculation is

$$\text{output} = \frac{\text{input} - \mu}{\sigma}.$$

3.3 Defining the Scheduler

The `diffusers` package allows us to take input images, add noise to them, and feed the new noisy images into the model.

The noise schedule used, as mentioned previously in Section 2.2.1, is based off a cosine function. In particular, the `squaredcos_cap_v2` within the `DDPMScheduler` class is used. Figure 3 shows the amount of noise added for both functions: the noise added by the cosine function is far more gradual across the entire training time-series. This effect is even more marked when scaling up $T = 4000$ from $T = 1000$, as seen in Figure 3e. As observed by Nichol et al. [21], using $T = 4000$ delivers improved results due to the gradual noise addition — the impact on the quality of the model can be seen in Figure 10 (Appendix B).

In particular, it is now possible to supply a value for the variance schedule offset s . s must provide an upper bound for $\sqrt{\beta_0}$. For normalised images between $[-1, 1]$ with discrete pixel values of up to 255, the bin size is $\frac{2}{255} = 0.00784 \approx 0.008 = s$. This corresponds to the experimental value [21] and used on implementation in [38].

Thus the schedule function (Equation 8) is

$$f(t) = \cos \left(\frac{\frac{t}{T} + 0.008}{1.008} \cdot \frac{\pi}{2} \right)^2.$$

3.4 Training the Model

The model is an adaptation of the Hugging Face *Introduction to Diffusers* model [39]. This MVP is a compromise between speed and image quality; initial inference samples can be found in Appendix B, Figure 10. The final code can be found at [40].

The core model used is a variant of a U-net. This involves downsampling an image, or halving the image size by 2, through each ResNet layer. This is then passed through the same number of upsampling layers to obtain the original image size. Here, the `UNet2DModel` provided in `diffusers` is used, which creates the desired architecture (Figure 4) in `PyTorch`.

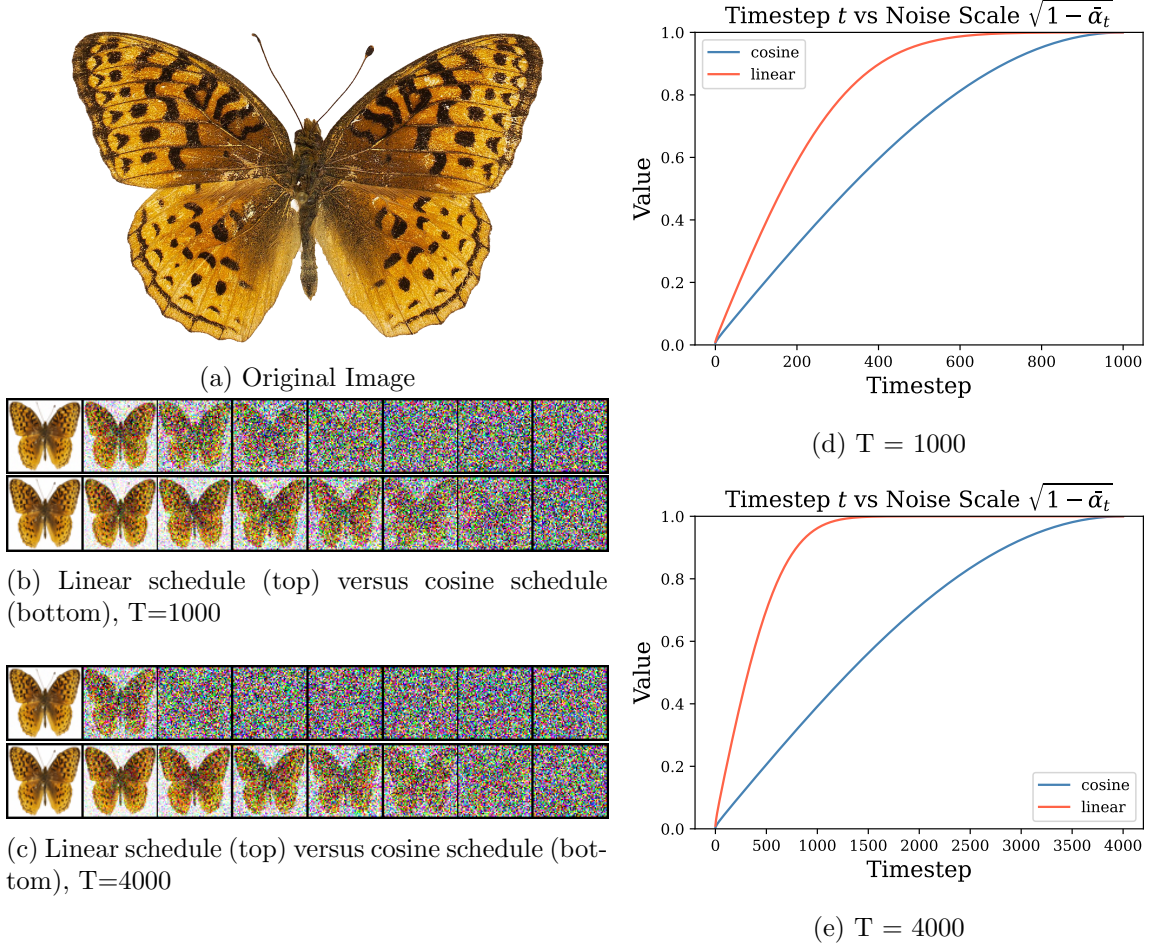


Figure 3: Linear vs. Cosine Variance Schedule Applied over 1000 and 4000 timesteps

The training is done over 30 epochs. In practice, it is possible to over-fit the model by increasing epochs too much. This is prevented by using the early-stopping method, which prevents wasting computation on minute marginal improvements in the model. When the absolute loss between two consecutive training loops is less than an specified tolerance ($1e^{-4}$), that epoch is terminated early.

Each data split of 1000 images is fed into the model as 20 batch sizes of 50. On a standard CPU, for 30 epochs, this takes about one hour training in total. With GPU integration (on a single NVIDIA GeForce RTX 3060, provided by the University of St Andrews School of Computer Science), training loops for image size of 64×64 pixels speed up to 1 minute per 5 epochs, allowing an entire training loop to run in 5 minutes.

Taking a log plot of loss across all training rounds for all iterations, the model has initial rapid improvement and then continues to get better at a slower (and inconsistent) rate, as shown in Figure 5. The trend of continual decline for 0% data in Figure 5a suggests that the model keeps learning as new data is provided at each round of training. This is not the case for the case of 100% data reinjection, where the slight increase in the log-loss trend suggests that, as the loss *increases* over time after the initial learning period, the model may begin to over-fit or collapse, consistent with [41].

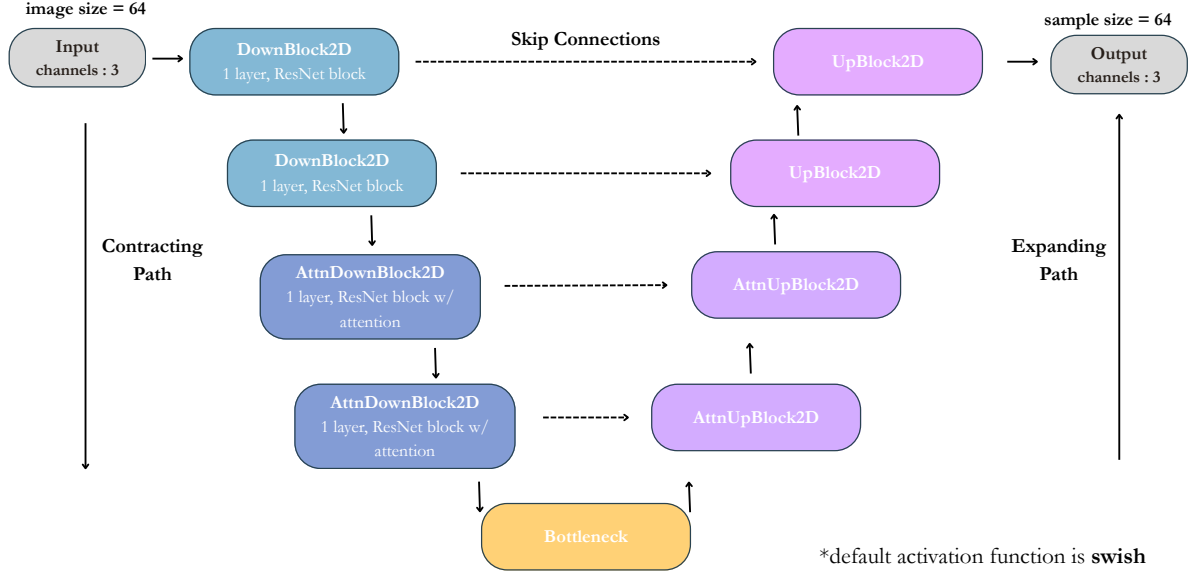


Figure 4: UNet Architecture

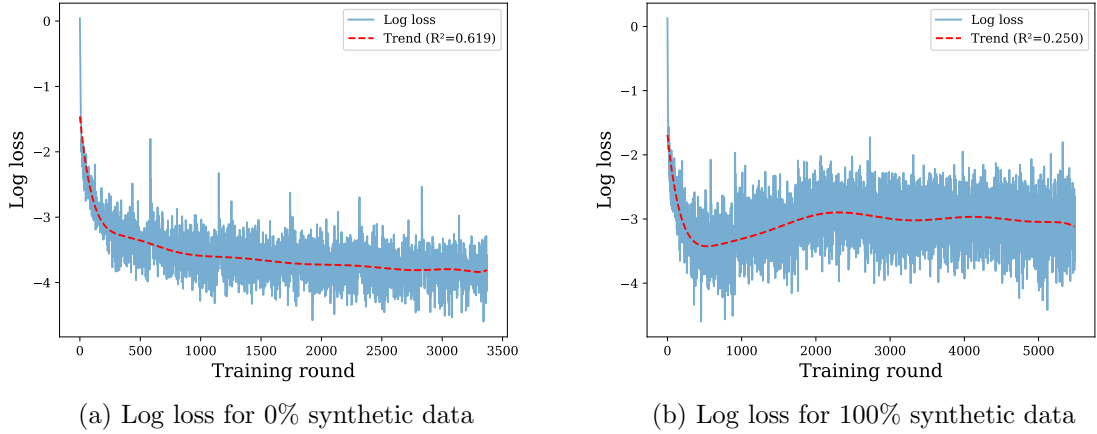


Figure 5: Log loss during training for different synthetic ratios

3.5 Sampling the Model

Once the model has been defined and trained, it is sampled to generate synthetic images. This is done by running through the scheduler time-steps in reverse (from T to 0). Each sample is updated T times, getting the model prediction at each step. In [21], random short-cuts (or ‘time-jumps’) are used to prevent having to go through every single time-step, without incurring significant loss in accuracy of predictions. The model implements this logic by sampling a random time-step for each image and getting the model prediction at that step.

The parameters for maximum time-steps T and image size are chosen based on the trade-off between speed and feature accuracy, particularly in the sampling stage. In Figure 10, one can see the effect of changing these parameters on the quality of samples taken after a single

iteration of training carried out on 0% synthetic data (all real). The combination that provides enough feature resolution, while retaining reasonable sampling times, is that of $T = 4000$, image size = 64. With these parameters, it takes approximately 12 minutes to sample and output 50 images to disk.

4 Provoking Model Collapse

4.1 Initial Findings

The initial model with hyperparameters $T = 4000$ and image size of 64 outputs images of which a sample of the first generation can be seen in Figure 10. The model is trained further on additional data, simulating the real-world process of retraining to improve the quality of model outputs. This means not training an entirely new model, but rather updating the weights learnt in previous runs. The dataset used for training each iteration containing synthetic data of varying ratios in the range 0% – 100%. The pseudocode for the initial round of experiments is given in Algorithm 1.

Algorithm 1 Iterative Training Pipeline with Synthetic Data Reinjection

```

for each synthetic ratio  $r \in \{0, 20, 40, 60, 80, 100\}$  do
  Initialise a fresh DDPM model  $\mathcal{M}$ 
   $\mathcal{G} \leftarrow \emptyset$  ▷ No generated images at the start
  while new real data  $\mathcal{R}$  is available do
    Sample all instances from  $\mathcal{G}$  ▷ Synthetic portion
    Sample  $1000 - \text{len}(\mathcal{G})$  instances from  $\mathcal{R}$  ▷ Real portion
     $\mathcal{D} \leftarrow \mathcal{R} \cup \mathcal{G}$  ▷ Combine into training set
     $\mathcal{M}.\text{train}(\mathcal{D})$ 
     $\mathcal{G} \leftarrow \mathcal{M}.\text{sample}(n = 1000 \times r)$  ▷ Create new synthetic pool
  end while
end for

```

Results for the final outputs after all rounds of training can be seen in Figure 6 (the maximum number of training iterations reached is 8). We observe the following from the outputs:

- At 0%, the image quality has improved relative to the initial model outputs. These are very clearly identifiable butterflies.
- At 20% and 40%, a few figures still resemble butterflies with defined features and contours. Noisy, ‘fuzzy’ images start to appear.
- From 60% to 100%, nearly all samples are near-uniform blocks of colour with unidentifiable forms. Collapse here is evident.

These images are used as a qualitative baseline with which to verify the trends understood with the metrics. The quantitative analysis of such results is given in the following section.

4.2 Calculating Metrics on Initial Results

FID taken from [42] is a PyTorch implementation of the metric as described in Section 2.4. All four feature values (64, 192, 768, 2048) are tested for the representativeness of results; the most indicative results were obtained at feature value of 192. The score is calculated using all sample images available per ratio and iteration. The sample number is limited by the compute available during image sampling step in the pipeline. Only the number of synthetic images needed for

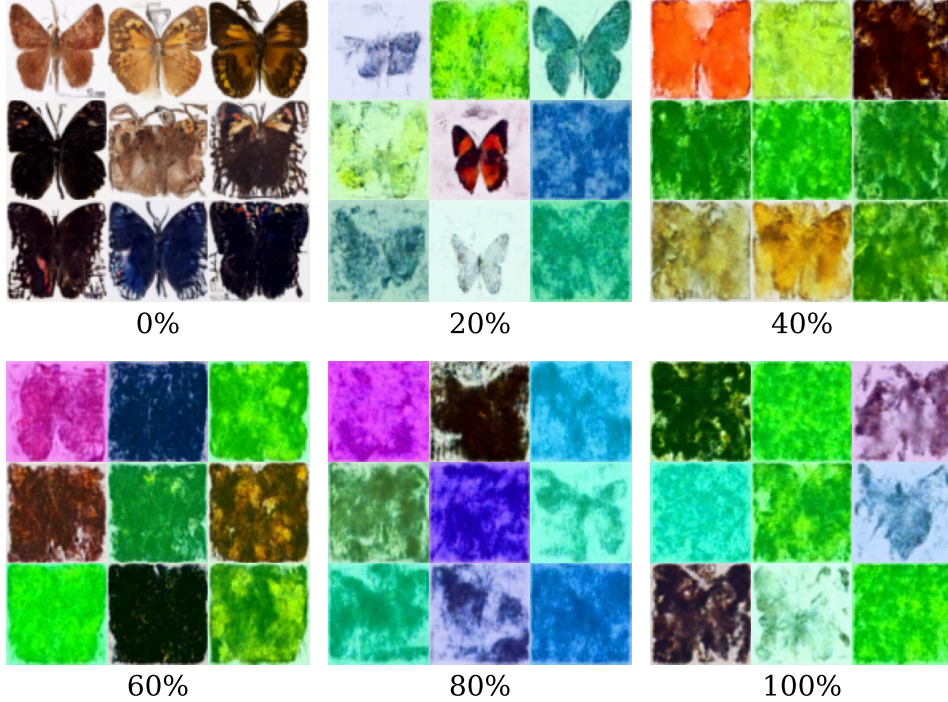


Figure 6: Images generated during iteration 5 for varying percentages (20% intervals) of synthetic data

training are generated per iteration. As such, there are more images to use in calculating the score for higher values of r .

FID scores relative to the real dataset (Figure 7a) increase for most synthetic ratios: the distributions progressively diverge with every iteration. The exception is for 0% synthetic data, where the FID score *decreases* (Figure 7b), confirming that the model improves when trained exclusively on real data — this consistent with qualitative results. At 100% the largest divergence is observed, with intermediate synthetic ratios also exhibiting increasing FID scores with every iteration.

When FID is computed relative to iteration-1 synthetic outputs, the score at iteration 1 is 0 for all values of r by definition (they are identical distributions). Note that this baseline confuses model improvement: the FID score increases for the 0% trial, despite the model improving: any distributional shift inflates the score. This baseline therefore captures divergence from initial generative quality but cannot distinguish improvement from collapse.

Throughout, the R^2 values indicate that trend-lines fit better on plots with a clear exponential increase in FID score, suggesting that reaching model collapse, therefore, is an exponential process. This aligns with the intuition that synthetic data quality degrades by compounding across training iterations: as each generation of outputs grows worse, the model trained on them deteriorates further still, accelerating the rate of divergence towards collapse.

To establish a collapse threshold from FID scores, the divergence is defined relative to two baselines: the real dataset, and the synthetic data at generation 1. Let F_α denote the conservative threshold for the real baseline, and F_β denote the more sensitive threshold of the synthetic baseline. Model collapse is the point at which the FID score exceeds both F_α and F_β . Using the Figures 7a and 7c, reasonable estimates for the two thresholds are $F_\alpha \approx 120$, $F_\beta \approx 50$. Under F_α , 40% data reinjection collapses around iteration 6, while 60%, 80%, and 100% collapse around iterations 3–4. Under F_β , all reinjection ratios above 40% exhibit collapse

by iteration 3–4. This suggests that the synthetic proportion that induces collapse is somewhere between 20% – 40%, not accounting for the exponential rise.

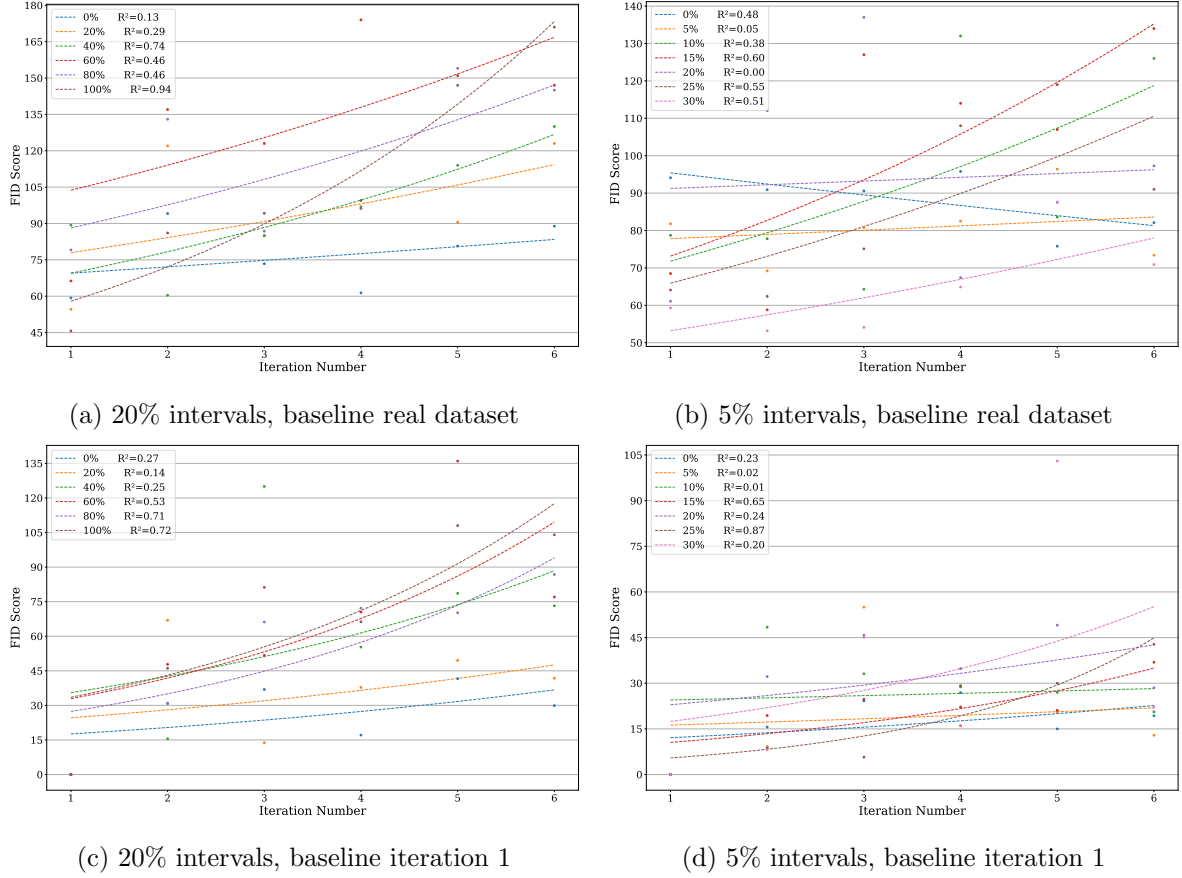


Figure 7: FID scores for varying % of synthetic data

The SSIM and MMD scores (Appendix C) are also calculated similarly, comparing each iteration of training outputs to both a real and synthetic baseline. SSIM does not suggest any clear relationship between synthetic ratio and change in SSIM. The trend is largely decreasing (Figure 11a, 11c) but there are inconsistencies, such as the increased trend for 20% synthetic baseline. The values also all condense to a narrow region, making it difficult to establish a threshold. Note that in the synthetic case, the first plot value is 1 by definition for all values, and thus removed as it makes it difficult to distinguish the remaining points — only iterations 2 to 6 are plotted.

The MMD plots (Figure 12a) show clear monotonic improvement for the 0% ratio relative to the real baseline. The 20% ratio also demonstrates a small decrease relative to the other plots. This drift across iterations is not visible in the other mixed-ratio trials, and instead scores remain constant. This suggests that the addition of synthetic data dominates the quality signal and forces collapse across iterations. The scores are generally more unstable relative to the synthetic baseline, but the best fits for ratios 40% and 100% are again exponential, confirming the previous hypothesis. Neither baselines permit the derivation of a suitable MMD threshold.

Carrying out PCA relative to the synthetic baseline on these results (Appendix D) demonstrates the initial hypothesis that model collapse is strongly related to decrease in the variance of the output distribution. If PCA by definition retains the features of maximum variance, the small, dense clouds indicate data grouped in a region of low variance. The sparse clouds at

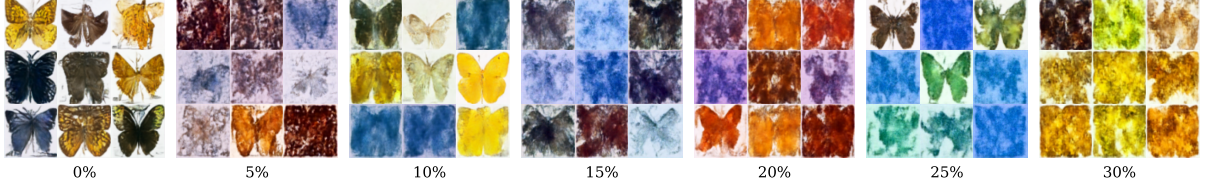


Figure 8: Images generated during iteration 5 for varying percentages (5% intervals) of synthetic data

iteration 1, and used in comparison in each plot, show higher variance and thus higher quality outputs with more diversity (these clouds are overlapping by definition as they are comparing the same data). In particular, it is visually notable that the clouds become denser by iteration 3 for the ratio 40%, whereas it takes later generations for the 20% ratio.

From these results, the following hypothesis is derived: the maximal ratio of synthetic reinjection for this particular model with the limited generations between 20% and 40%. The next section tests this hypothesis by using a smaller granularity of interval.

4.3 Further Trials

To obtain a more precise understanding of the point of collapse, more runs are conducted between 0% and 30% at 5% granularity. A sample of images from these intervals is given in Figure 8. Visually, from the occurrence of butterflies at synthetic ratio 25%, this suggests it is a stable ratio at which collapse is not yet obtained. There are inconsistencies in the outputs, with 5%, 15% and 20% reinjection showing surprisingly high drift.

The FID scores relative to the real data confirm some of these observations. Firstly, the 0% trend decreases as previously (Figure 7b), whereas the rate of decay for the other percentages is still exponential for the best fits, even at such low levels of synthetic data. The FID score for 15% significantly underperforms relative to the other values. Breaches of the suggested threshold F_α occur occasionally, but never for the ratios of 25% and 30%. The points relative to the synthetic baseline are also mostly below the threshold of F_β , with the ratio of 25% in particular being well below it, and exhibiting a strong and reliable trend.

To conduct a more in-depth understanding of the stability of these ratios, in particular those between 0% – 25% versus 30%, the variance is taken across the FID and MMD scores. Levene’s test⁶ carried out on the variances suggest that the differences in variance between some of the scores are statistically significant. Looking into the standard deviations, the FID spike for 30% with synthetic baseline contrasts with the much lower score for 25%, which also has the joint minimum variance for MMD. This further makes the case for 25% as the stability ceiling.

⁶This test is used to test if k samples have equal variance, or if their variances are statistically different.[43]

Table 1: Standard deviation of FID and MMD scores across iterations (ratios 0–30), for FID iteration 1 excluded as a near-zero artefact. Italicised denotes minimum, bold denotes maximum SD per column.

Ratio	FID SD ($\downarrow$)		MMD SD ($\downarrow$)	
	<i>Synthetic</i>	<i>Real</i>	<i>Synthetic</i>	<i>Real</i>
0	<i>5.24</i>	<i>7.70</i>	0.0527	0.0509
5	18.52	9.34	0.0116	0.0068
10	10.42	28.12	0.0033	0.0083
15	7.00	31.00	0.0043	0.0092
20	8.92	28.34	0.0054	0.0022
25	15.67	23.46	<i>0.0031</i>	<i>0.0021</i>
30	38.38	12.90	0.0044	0.0053

Observing PCA across two dimensions for the 5% granularity (Figure 14), the plots for 15% and 20% are particularly dense, whereas the clouds become sparser for 25% before collapsing again at 30%. This further evidences some unpredictable behaviour within smaller ranges, but stability against collapse at the ratio of 25%.

Therefore it is possible to propose the ratio of 25% as the maximal stable ratio for the following reasons:

1. Threshold compliance: some intermediary ratios behave unpredictably, but ratio 25% remains within the threshold $F_\alpha \approx 120$ for all iterations. All ratios 0% – 25% remain within the threshold $F_\beta \approx 50$, whereas 30% exceeds this point.
2. Variance: The 25% ratio displays the minimum MMD standard deviation, and a stable FID variance relative to the spike at 30%.
3. PCA: the feature clouds remain relatively spread for 25%, whereas they become notably denser by 30%.

5 Evaluation

This section seeks to evaluate the successes and limitations of the model chosen, as well as of the measures themselves in indicating the degree of the drift of the synthetic distribution and the point at which collapse has been reached. It further discusses some of the irregularities observed in the data, and potential improvements to the project to further test the robustness of the hypothesis.

The model used is intentionally an MVP DDPM model. It allows for relative mathematical tractability, and the ability to control the environment, inputs and training tightly. While the forward process is mathematically defined, the reverse process is still somewhat a ‘black-box’, making it difficult to understand concretely at which point, and *why* collapse happens. Further, the architecture choices in the very structure of the UNet impact the quality of synthetic outputs, independently from the synthetic input ratio. The chosen iterative MVP, alongside its compounding behaviour (rather than ‘from scratch’), directly impacts the exponential rate of collapse. It would be beneficial to test the proposed threshold ratio of 25% across different image sizes, time-steps and UNet architectures to understand if this ratio is maximal across varied parameters.

The training data is extremely domain-specific, which ensures collapse is captured across the entirety of the model in its single function of outputting representations of butterflies. However,

this restricts output diversity from the inception of the model, which narrowly focusses the possible metrics and the behaviour that they capture. The cleaning and sample prep over the dataset deliberately restrict the original images, at least visually. While chosen for their easily identifiable form, the butterfly dataset is ultimately small by stable diffusion standards. This could potentially infringe on the ability to distinguish the lower granularity ratio results from noise. One cleansing technique of the original dataset that is not carried out is duplicate removal, on the basis that transformations and background removal may shift these images, even minutely, preserving their utility. Were there to be duplicate images in the original data, it would reduce further the size of the dataset and the information it encodes, hindering the potential improvement of the model over training rounds. In the real world, there are many more augmentation techniques possible such as blurring, sharpening, random cropping or patching, which could aid in extending the dataset.

The measurements are selected based on their widespread use. FID and MMD (with its KID implementation) provide insight into the relationship of the underlying statistical distributions rather than purely structural pixel relationships, as SSIM does. Combining this with PCA, this provides a range of perspectives with which to understand the potential points of collapse. The joint agreement between FID results and KID, which is supposedly sample-size-unbiased, together provide strong indication of the distribution collapse. An important limitation of FID and KID is that Inception-v3 features are ImageNet-trained and may be misaligned with the specific domain and feature profile of the data used here. Indeed, the negative scores obtained for both FID and KID suggest numerical instability, potentially caused by the small sample sizes. Given that sample sizes also change based on the ratio value, hence on the availability of the synthetic data, the scores are more reliable for larger r values. A simple solution to this would be to provide confidence intervals around both metrics; this would allow threshold comparisons to be stated probabilistically rather than pass/fail. The scores themselves could be normalised against the proposed threshold to produce a dimensionless ‘collapse index’ that would be comparable across different parameters.

An important limitation of the project is the heuristic, rather than statistical, proposal of the threshold values. There are no established benchmarks for the FID scores with synthetic baselines, and with a certain degree of perceptual tolerance. The FID benchmarks of F_α, F_β are based on a single individual’s intuition. This could be improved by either a perceptual study, or the application of a pre-trained image classification model to test for the recognition of a ‘butterfly’ within an image. Collapse could be identified as the point in which fewer than $x\%$ of images are classified as ‘butterflies’. This would further emphasise the perceptual function of Gen AI models.

Finally, further experiments and techniques can be used to test the hypothesis of a stable ratio at 25% reinjection. For instance, reducing the granularity between the 20% and 30% intervals would locate the collapse-inducing ratio more precisely. Importantly, testing the 25% ratio on iterations beyond the current restrictions would strengthen the case for stability. This can only be done with a sufficiently large dataset (for example, to train on 25% synthetic reinjection for 10 iterations would require 7,750 real images). Using an FID-threshold induced early-termination can reduce the number of runs if collapse is reached.

6 Conclusion

The real world corpus of the Smithsonian’s collection of butterflies is marked by variety in aesthetic and origin, with true diversity. The use of Gen AI, ultimately a refined form of mimicry, will continue to increase, particularly as these models become more capable. This study demonstrated that the question of collapse is not concerned by the quality of the initial

model, but rather depends on the quality of the inputs to the models (determined by the proportion of real data) and the number of iterations. The collapse of our Gen AI models is not of immediate concern – GPT5.2 will not collapse after one round of training – but it is the progressive drift of the synthetic outputs that will cumulatively degrade models. In this light, generative models will not expand and embellish our Internet, but rather increasingly saturate it by repetition, and subtly less perfect butterflies.

Appendices

A Dataset Examples



Figure 9: Sample Images from Dataset

B Timestep and Image size tests



Figure 10: Trial of Images Generated for Iteration 1 of Training on Non-Synthetic data from dataset [36], varying Timesteps and Original Image Pixel Size

C Scores for SSIM, FID, MMD

C.1 FID Results

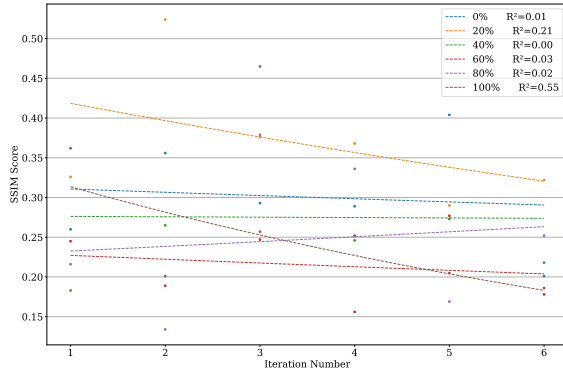
Table 2: FID Scores ($\downarrow$, features = 192) by Ratio and Iteration

Int.	Ratio	Synthetic Baseline Iterations						Real Baseline Iterations					
		1	2	3	4	5	6	1	2	3	4	5	6
20%	0	-2.40×10^{-6}	30.9	36.9	17.1	41.6	29.9	59.3	94.1	73.4	61.4	80.7	88.9
	20	5.37×10^{-9}	66.9	13.8	37.8	49.5	41.8	54.6	122	84.9	96.2	90.5	123
	40	1.20×10^{-10}	15.5	125	55.3	78.6	73.2	89.3	60.4	85.0	96.9	114	130
	60	-1.48×10^{-10}	47.8	51.7	70.5	136	77.0	66.3	137	123	174	151	147
	80	9.79×10^{-11}	30.8	66.2	72.2	70.2	86.8	79.1	133	86.8	96.2	154	145
	100	-5.59×10^{-11}	46.1	81.2	66.3	108	104	45.7	86.1	94.2	99.5	147	171
5%	0	-3.10×10^{-6}	15.6	24.2	26.9	15.0	19.3	94.1	90.9	90.6	95.8	75.8	82.1
	5	-3.84×10^{-6}	9.15	55.0	16.0	20.5	12.9	81.8	69.2	80.8	82.5	96.4	73.4
	10	-2.51×10^{-6}	48.4	33.1	28.8	27.0	20.6	78.7	77.8	64.3	132	83.6	126
	15	-1.04×10^{-6}	19.4	24.7	22.2	21.1	36.9	68.5	58.8	127	114	107	134
	20	1.91×10^{-8}	32.2	45.8	34.8	49.1	28.5	61.1	112	137	67.4	87.6	97.3
	25	-1.01×10^{-9}	8.71	5.70	29.2	30.0	42.8	64.1	62.4	75.1	108	119	91.0
	30	3.43×10^{-10}	8.15	45.1	16.1	103	22.2	59.3	53.2	54.1	64.9	87.5	70.9

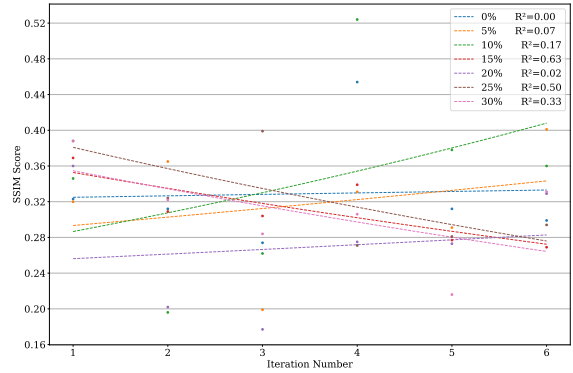
C.2 SSIM Results and Plots

Table 3: SSIM Scores ($\uparrow$) by Ratio and Iteration

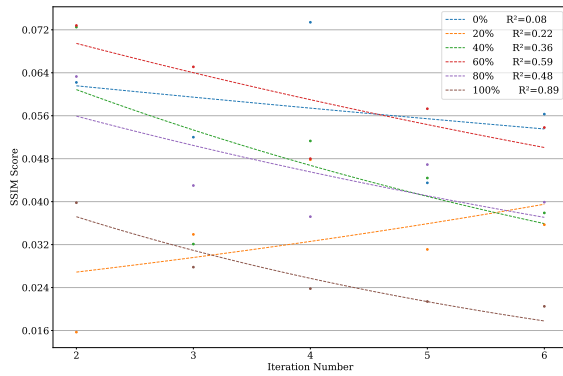
Int.	Ratio	Synthetic Baseline Iterations						Real Baseline Iterations					
		1	2	3	4	5	6	1	2	3	4	5	6
20%	0	1.000	0.0622	0.0520	0.0734	0.0435	0.0563	0.260	0.356	0.293	0.289	0.404	0.201
	20	1.000	0.0157	0.0339	0.0478	0.0311	0.0357	0.326	0.524	0.376	0.368	0.290	0.322
	40	1.000	0.0725	0.0321	0.0513	0.0444	0.0379	0.183	0.265	0.465	0.246	0.273	0.218
	60	1.000	0.0728	0.0651	0.0480	0.0573	0.0538	0.245	0.189	0.247	0.156	0.277	0.178
	80	1.000	0.0633	0.0430	0.0372	0.0469	0.0399	0.216	0.134	0.379	0.336	0.169	0.252
	100	1.000	0.0398	0.0278	0.0238	0.0214	0.0205	0.362	0.201	0.257	0.252	0.205	0.186
5%	0	1.000	0.0591	0.0496	0.0492	0.0546	0.0505	0.323	0.312	0.274	0.454	0.312	0.299
	5	1.000	0.0580	0.0511	0.0381	0.0405	0.0611	0.320	0.365	0.199	0.331	0.291	0.401
	10	1.000	0.0609	0.0593	0.0385	0.0472	0.0499	0.346	0.196	0.262	0.524	0.378	0.360
	15	1.000	0.0570	0.0461	0.0553	0.0547	0.0556	0.369	0.309	0.304	0.339	0.277	0.269
	20	1.000	0.0551	0.0417	0.0361	0.0455	0.0434	0.360	0.202	0.177	0.275	0.273	0.329
	25	1.000	0.0565	0.0550	0.0449	0.0429	0.0384	0.388	0.324	0.399	0.271	0.281	0.294
	30	1.000	0.0632	0.0484	0.0594	0.0261	0.0445	0.388	0.322	0.284	0.306	0.216	0.331



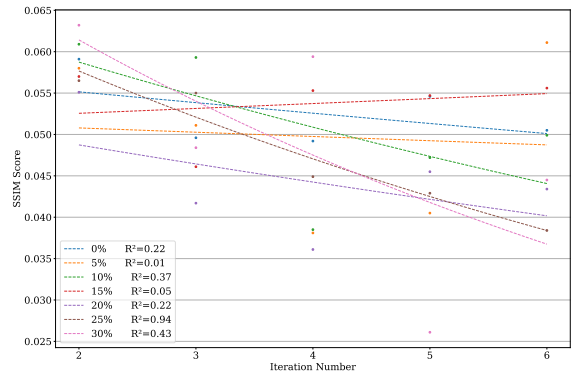
(a) 20% intervals, baseline real dataset



(b) 5% intervals, baseline real dataset



(c) 20% intervals, baseline iteration 1



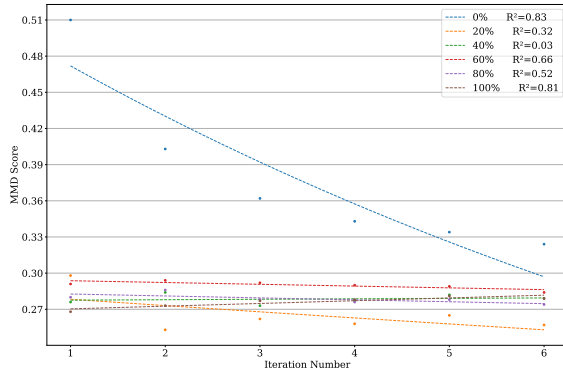
(d) 5% intervals, baseline iteration 1

Figure 11: SSIM scores for varying % of synthetic data

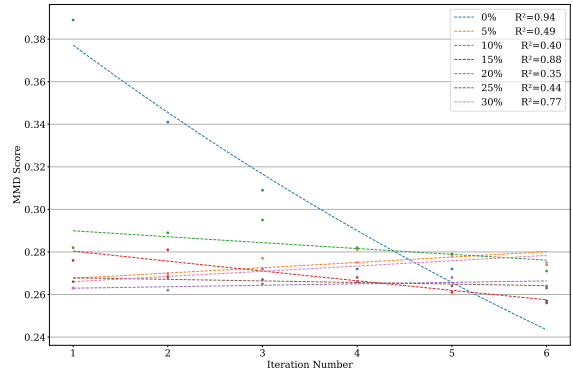
C.3 MMD Results and Plots

Table 4: MMD (KID) Scores ($\downarrow$) by Ratio and Iteration

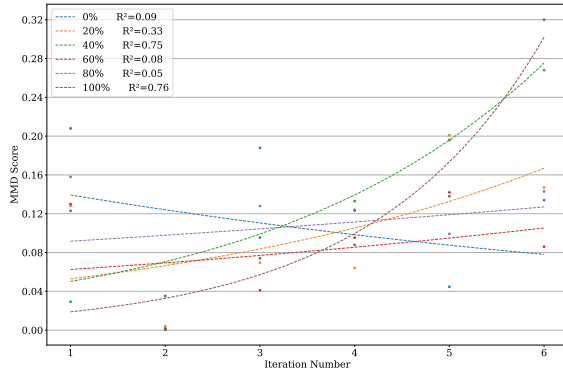
Int.	Ratio	Synthetic Baseline Iterations						Real Baseline Iterations					
		1	2	3	4	5	6	1	2	3	4	5	6
20%	0	0.208	-0.0352	0.0954	0.124	0.0447	0.134	0.510	0.403	0.362	0.343	0.334	0.324
	20	0.128	-0.00400	0.0694	0.0642	0.201	0.147	0.298	0.253	0.262	0.258	0.265	0.257
	40	0.0293	-0.00180	0.188	0.133	0.196	0.268	0.276	0.284	0.273	0.277	0.282	0.279
	60	0.130	-0.000971	0.0412	0.0954	0.142	0.0860	0.291	0.294	0.292	0.290	0.289	0.284
	80	0.158	-0.000698	0.128	0.123	0.0992	0.143	0.280	0.286	0.278	0.276	0.278	0.274
	100	0.123	-0.000608	0.0741	0.0879	0.138	0.320	0.268	0.273	0.277	0.278	0.281	0.279
5%	0	-0.0151	0.0314	0.0822	0.101	0.102	0.125	0.389	0.341	0.309	0.272	0.272	0.256
	5	0.0904	0.0811	0.0737	0.0643	0.0609	0.0640	0.263	0.269	0.277	0.281	0.279	0.274
	10	0.0464	0.0438	0.0491	0.0483	0.0521	0.0525	0.282	0.289	0.295	0.282	0.279	0.271
	15	0.0400	0.0393	0.0366	0.0402	0.0435	0.0491	0.276	0.281	0.272	0.266	0.261	0.257
	20	0.0403	0.0380	0.0479	0.0465	0.0489	0.0523	0.263	0.262	0.265	0.266	0.268	0.264
	25	0.0431	0.0425	0.0394	0.0431	0.0457	0.0485	0.266	0.268	0.267	0.268	0.264	0.263
	30	0.0408	0.0390	0.0402	0.0386	0.0462	0.0495	0.263	0.270	0.272	0.275	0.278	0.275



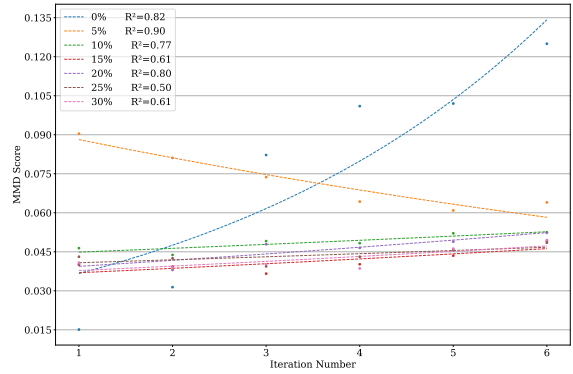
(a) 20% intervals, baseline real dataset



(b) 5% intervals, baseline real dataset



(c) 20% intervals, baseline iteration 1



(d) 5% intervals, baseline iteration 1

Figure 12: MMD scores for varying % of synthetic data

D PCA Results for 20% intervals of synthetic data

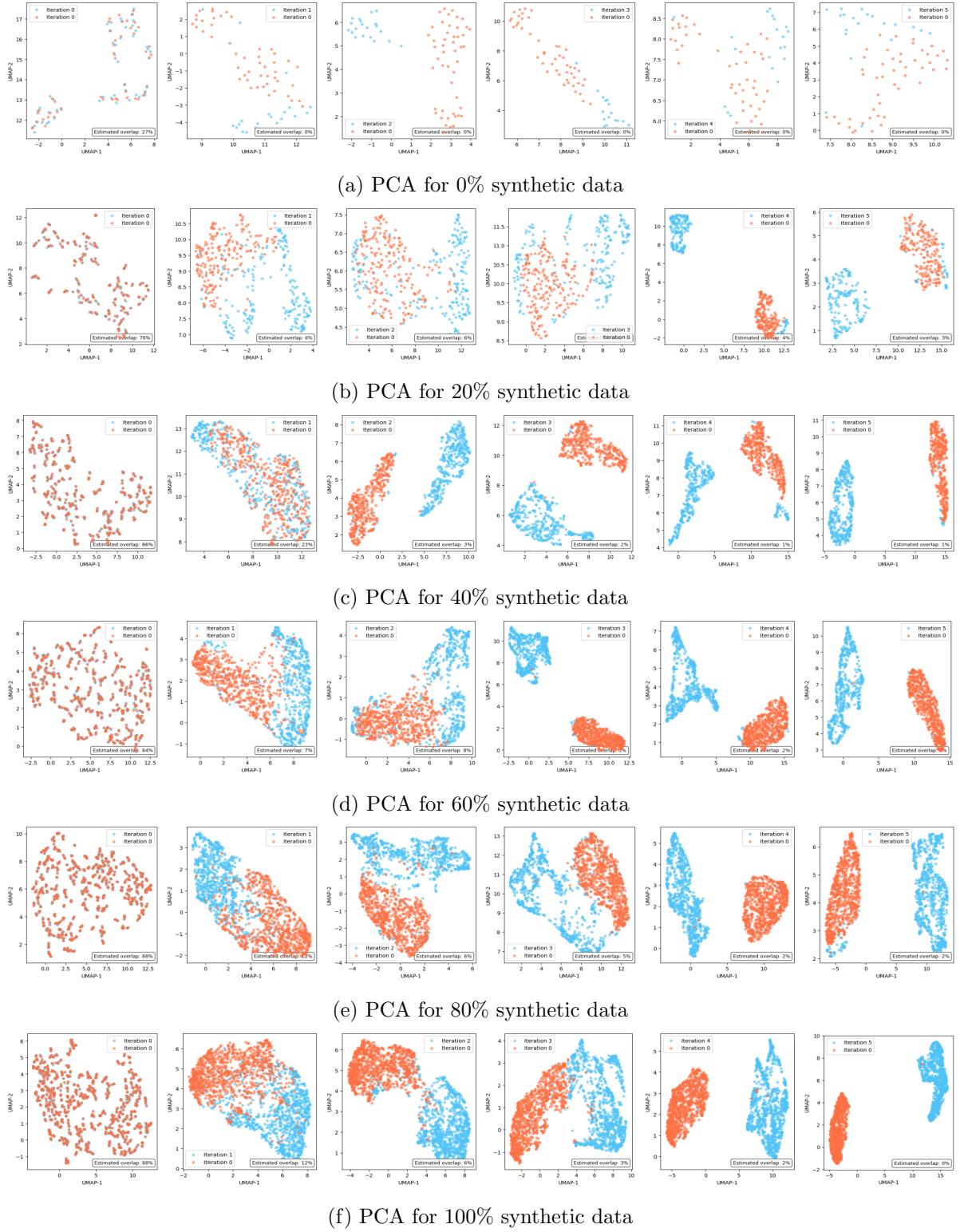
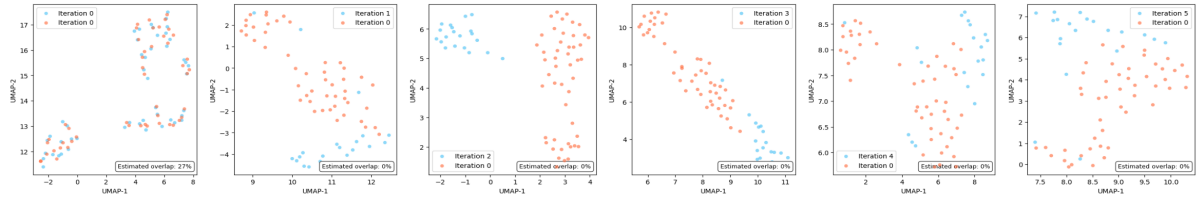
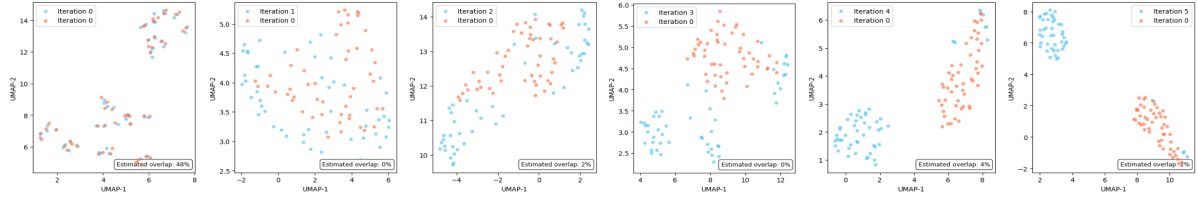


Figure 13: PCA results for 20% intervals of synthetic data

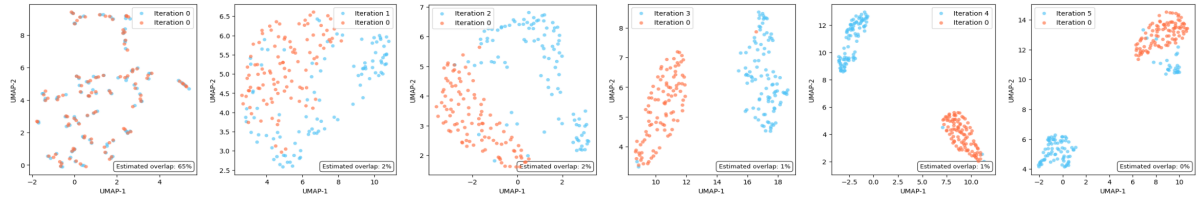
E PCA Results for 5% intervals of synthetic data



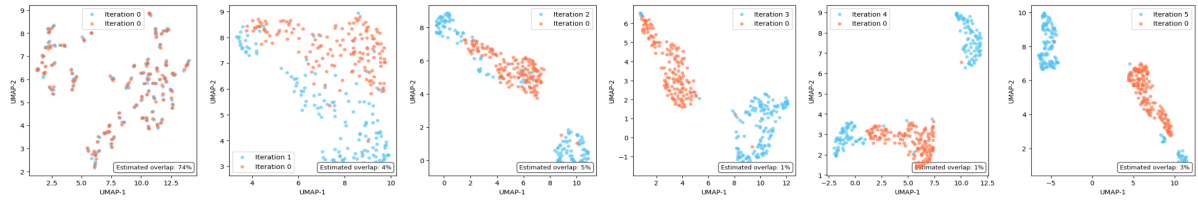
(a) PCA for 0% synthetic data



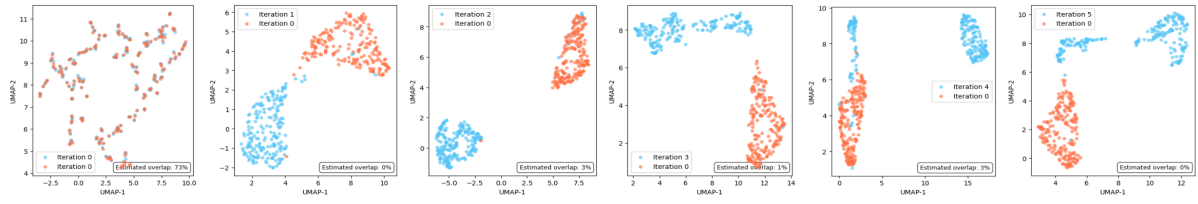
(b) PCA for 5% synthetic data



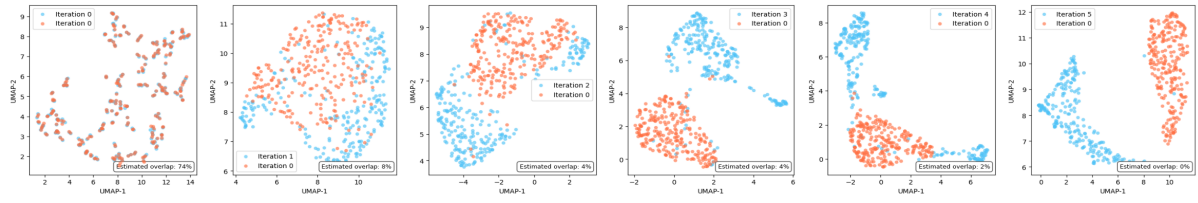
(c) PCA for 10% synthetic data



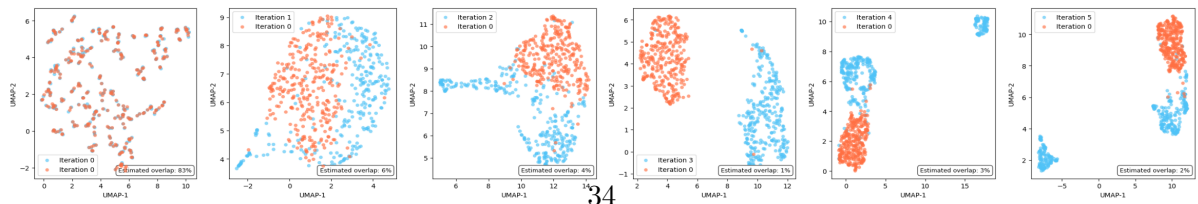
(d) PCA for 15% synthetic data



(e) PCA for 20% synthetic data



(f) PCA for 25% synthetic data



(g) PCA for 30% synthetic data

Figure 14: PCA results for 5% intervals of synthetic data

References

- [1] OpenAI, *Dall-e now available without waitlist*, Sep. 2022. [Online]. Available: <https://openai.com/index/dall-e-now-available-without-waitlist/>.
- [2] A. Valyaeva, *Ai image statistics: How much content was created by ai*, Aug. 2023. [Online]. Available: <https://journal.everypixel.com/ai-image-statistics>.
- [3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2022, pp. 10 674–10 685. DOI: [10.1109/CVPR52688.2022.01042](https://doi.org/10.1109/CVPR52688.2022.01042). [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CVPR52688.2022.01042>.
- [4] T. Brown et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- [5] M. Kapuściński, *Gpt-5 training data: Evolution, sources, and ethical concerns — ttms*, Nov. 2025. [Online]. Available: <https://ttms.com/gpt-5-training-data-evolution-sources-and-ethical-concerns/>.
- [6] A. Kortylewski et al., *Can synthetic faces undo the damage of dataset bias to face recognition and facial landmark detection?* 2019. arXiv: [1811.08565](https://arxiv.org/abs/1811.08565) [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1811.08565>.
- [7] D. Herel and T. Mikolov, “Collapse of self-trained language models,” in *The Second Tiny Papers Track at ICLR 2024, Tiny Papers @ ICLR 2024, Vienna, Austria, May 11, 2024*, OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=DeuBfEWyR4>.
- [8] Y. Duan and S. Huang, “Empirical validation and measurement of the collapse phenomenon,” Jul. 2025. DOI: [10.13140/RG.2.2.24525.14562](https://doi.org/10.13140/RG.2.2.24525.14562).
- [9] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., ser. Proceedings of Machine Learning Research, vol. 37, Lille, France: PMLR, Jul. 2015, pp. 2256–2265. [Online]. Available: <https://proceedings.mlr.press/v37/sohl-dickstein15.html>.
- [10] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 6840–6851. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf.
- [11] Z. Shen, M. Mao, and P. Fan, “A primary comparison of diffusion models and generative adversarial networks for image synthesis,” in *Proceedings of the 2024 7th International Conference on Machine Learning and Machine Intelligence (MLMI)*, ser. MLMI ’24, Association for Computing Machinery, 2024, pp. 225–234, ISBN: 9798400717833. DOI: [10.1145/3696271.3696307](https://doi.org/10.1145/3696271.3696307). [Online]. Available: <https://doi.org/10.1145/3696271.3696307>.
- [12] M. Bohacek and H. Farid, “Nepotistically trained generative image models collapse,” in *ICLR 2025 Workshop on Navigating and Addressing Data Problems for Foundation Models*, 2025. [Online]. Available: <https://openreview.net/forum?id=mkZB0fKLX8>.

- [13] E. Dohmatob, Y. Feng, A. Subramonian, and J. Kempe, “Strong model collapse,” in *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, Eds., vol. 2025, 2025, pp. 15 656–15 691. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/284afdc2309f9667d2d4fb9290235b0c-Paper-Conference.pdf.
- [14] S. Alemohammad et al., “Self-consuming generative models go mad,” in *ICLR 2024*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/ebc042e767de551803ccfcc45e2454f5-Paper-Conference.pdf.
- [15] R. Hataya, H. Bao, and H. Arai, “Will large-scale generative models corrupt future datasets?” In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 20 498–20 508. DOI: [10.1109/ICCV51070.2023.01879](https://doi.org/10.1109/ICCV51070.2023.01879).
- [16] G. Martínez, L. Watson, P. Reviriego, J. A. Hernández, M. Juárez, and R. Sarkar, *Combining generative artificial intelligence (ai) and the internet: Heading towards evolution or degradation?* 2023. arXiv: [2303.01255](https://arxiv.org/abs/2303.01255) [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2303.01255>.
- [17] G. Martínez, L. Watson, P. Reviriego, J. A. Hernández, M. Juárez, and R. Sarkar, “Towards understanding the interplay of generative artificial intelligence and the internet,” in *Epistemic Uncertainty in Artificial Intelligence*, F. Cuzzolin and M. Sultana, Eds., Cham: Springer Nature Switzerland, 2024, pp. 59–73, ISBN: 978-3-031-57963-9.
- [18] L. Yang, S. Ding, Y. Cai, J. Yu, J. Wang, and Y. Shi, “Guidance with spherical gaussian constraint for conditional diffusion,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24, Vienna, Austria: JMLR.org, 2024.
- [19] C. Do, *The Multivariate Gaussian Distribution*. Stanford University, Oct. 2008. [Online]. Available: <https://cs229.stanford.edu/section/gaussians.pdf>.
- [20] J. H. Park, *From noise to clarity: The math behind denoising diffusion models*, Jul. 2025. [Online]. Available: <https://medium.com/@jpark7/from-noise-to-clarity-the-math-behind-denoising-diffusion-models-cf72900905c7>.
- [21] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *Proceedings of the 38th International Conference on Machine Learning*, M. Meila and T. Zhang, Eds., ser. Proceedings of Machine Learning Research, vol. 139, PMLR, Jun. 2021, pp. 8162–8171. [Online]. Available: <https://proceedings.mlr.press/v139/nichol21a.html>.
- [22] Z. Nasir, *Cosine-beta schedule for denoising diffusion models*, Dec. 2023. [Online]. Available: <https://www.zainnasir.com/blog/cosine-beta-schedule-for-denoising-diffusion-models/>.
- [23] S. J. Prince, *Understanding Deep Learning*. The MIT Press, 2023. [Online]. Available: <http://udlbook.com>.
- [24] N. Rogge and K. Rasul, *The annotated diffusion model*, Jun. 2022. [Online]. Available: <https://huggingface.co/blog/annotated-diffusion>.
- [25] K. P. Murphy, *Probabilistic Machine Learning : an Introduction*. Cambridge, Massachusetts: The MIT Press, 2022, ISBN: 9780262046824.
- [26] S. Galli, *Mutual information with python*, Aug. 2022. [Online]. Available: <https://www.blog.trainindata.com/mutual-information-with-python/>.
- [27] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. “O’Reilly Media, Inc.”, Sep. 2019, ISBN: 9781492032595.

- [28] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>.
- [29] J. Soch, *Kullback-leibler divergence for the multivariate normal distribution*, May 2020. [Online]. Available: <https://statproofbook.github.io/P/mvn-kl.html>.
- [30] J. Nilsson and T. Akenine-Möller, *Understanding ssim*, 2020. arXiv: 2006.13846 [eess.IV]. [Online]. Available: <https://arxiv.org/abs/2006.13846>.
- [31] A. Mallasto and A. Feragen, “Learning from uncertain curves: The 2-wasserstein metric for gaussian processes,” in *Advances in Neural Information Processing Systems*, I. Guyon et al., Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/7a006957be65e608e863301eb98e1808-Paper.pdf.
- [32] J. Brownlee, *How to implement the frechet inception distance (fid) for evaluating gans*, Aug. 2019. [Online]. Available: <https://machinelearningmastery.com/how-to-implement-the-frechet-inception-distance-fid-from-scratch/>.
- [33] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, no. 25, pp. 723–773, 2012. [Online]. Available: <http://jmlr.org/papers/v13/gretton12a.html>.
- [34] C. Cinarel, *Smithsonian butterflies*, 2026. [Online]. Available: https://huggingface.co/datasets/ceyda/smithsonian_butterflies.
- [35] X. Qin, H. Dai, X. Hu, D.-P. Fan, L. Shao, and L. Van Gool, “Highly accurate dichotomous image segmentation,” in *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XVIII*, Tel Aviv, Israel: Springer-Verlag, 2022, pp. 38–56, ISBN: 978-3-031-19796-3. DOI: 10.1007/978-3-031-19797-0_3. [Online]. Available: https://doi.org/10.1007/978-3-031-19797-0_3.
- [36] S. Mortarotti, *Full_butterfly_dataset_cleaned*, 2026. [Online]. Available: https://huggingface.co/datasets/stella-mj/full_butterfly_dataset_cleaned.
- [37] TensorFlow, *Data augmentation*, Jul. 2024. [Online]. Available: https://www.tensorflow.org/tutorials/images/data_augmentation.
- [38] huggingface, *Hugging face, diffusers package, scheduling method*, 2025. [Online]. Available: https://github.com/huggingface/diffusers/blob/main/src/diffusers/schedulers/scheduling_ddpm.py.
- [39] H. Face, *Introduction to diffusers*, 2022. [Online]. Available: <https://huggingface.co/learn/diffusion-course/unit1/2>.
- [40] stella-mj, *Github - stella-mj/mt4796-ddpm*, Apr. 2026. [Online]. Available: https://github.com/stella-mj/MT4796%5C_DDPM.
- [41] S. Qiu, A. Agarwala, J. Pennington, and L. Xiao, “Scaling collapse reveals universal dynamics in compute-optimally trained neural networks,” in *OPT 2024: Optimization for Machine Learning*, 2024. [Online]. Available: <https://openreview.net/forum?id=SVwnbepFsc>.
- [42] L. AI, *Frechet inception distance (fid)*, 2026. [Online]. Available: https://lightning.ai/docs/torchmetrics/stable/image/frechet_inception_distance.html.
- [43] NIST, *1.3.5.10. levene test for equality of variances*, 2019. [Online]. Available: <http://www.itl.nist.gov/div898/handbook/eda/section3/eda35a.htm>.